

0. 存储器基础

1. 位置 (Location)

CPU 内存寄存器

内存寄存器: 通常等同于**主存**, 但也包括**处理器内部的寄存器**以及**高速缓存**。

外部存储器: 由外围存储设备 (如光盘、磁盘、磁带) 组成, 处理器需要通过 I/O 模块来访问它们。

2. 容量 (Capacity)

对于内存存储器, 容量通常使用字或字节 (1 字节 = 8 位) 来表示, 常见的字长有 8 位、16 位和 32 位。由字长 (天然组织单位) 和字数共同决定, 总容量 = 字长 × 字数;

对于外部存储器, 容量一般直接用字节来表示。

3. 传输单位 (Unit of Transfer)

字 (Word): 存储器组织的“天然”单位, 其大小通常等于表示一个整数或指令长度的位数。

可寻址单元: 有些系统是字级别的, 有些允许在字节级别上寻址。地址位数 A 与可寻址单元个数 N 之间的关系表现为 $2^A = N$ 。

传输单位: 对主存, 是指一次读出或写入存储器的位数。对外部存储器, 数据通常以大得多的单位进行传输, 称为“**块**”。

4. 访问方式 (Access Method) 见第 5 点

5. 性能 (Performance) 见第 6 点

6. 物理类型与特性 (Physical Type & Characteristics)

物理类型: 目前最常见的是半导体存储器、磁性表面存储器, 以及光学和磁光的存储器。

易失性/非易失性: 易失性存储器在断电时信息会丢失 (如部分半导体存储器), 而非易失性存储器 (如磁性表面存储器) 在断电后仍能保留信息。

可擦除/不可擦除: 不可擦除存储器 (如只读存储器 ROM) 的内容无法被更改, 这种存储器必须是非易失性的。

7. 组成 (Organization)

在随机访问存储器中, “组成”特指构成数据的位的物理排列方式。

1. Memory hierarchy 多级存储器

存储器系统按照「速度-容量-成本」的平衡原则构建分层架构, 从靠近 CPU 到远离 CPU 依次为: CPU 寄存器 → L1 Cache → L2 Cache → L3 Cache → 主存 → 磁盘缓存 → 磁盘 → 光盘 → 磁带。

核心规律: 沿层级向下, 每位存储成本递减、总容量递增、访问速度递减, 设计目的是让存储系统兼具高速存储的访问速度与大容量低速存储的成本优势。

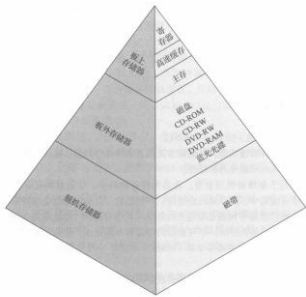


图 4-1 存储器层次结构

2. Locality of reference 局部性原理

是 Cache 能够有效提升性能的核心理论基础, **核心特征为内存访问倾向于聚集在局部区域**:

- **时间局部性**: 近期被访问过的内存位置, 短期内大概率会被再次访问 (如循环迭代中的变量)。
- **空间局部性**: 访问某一内存地址后, 其相邻的地址也大概率

会被访问 (如顺序执行的指令、遍历数组)。
动态特征: 短期内, 内存访问的聚集区域保持固定; 长期来看, 访问聚集区域会发生切换变化。

3. Characteristics of hierarchy memory system 多级存储器的特点

(1) 存储器通用特性维度 见第 0 点

(2) 多层次的核心特点

- 层级越靠近 CPU, 速度越快、容量越小、单位成本越高;
- 上层存储是下层存储的子集, 数据以块为单位逐级缓存;
- 依托局部性原理, 程序的绝大多数访问都命中高速层级, 整体平均访问速度接近高速存储。

4. Access time of multilevel memory system Calculation and diagrams 多级存储器访问时间的计算和图形示意

(1) 计算公式 (两级存储器) **平均访问时间公式**:

$$T = T_1H + (T_1 + T_2)(1-H)$$

- T_1 : 上层存储器 (如 Cache) 的访问时间

- T_2 : 下层存储器 (如主存) 的访问时间

- H: 上层存储器的命中率

3 级: $T_{avg2} = H_1T_1 + (1-H_1)(T_1 + T_{avg2})$

(2) 图形示意 **例题 P3**

以命中率为横轴、平均访问时间为纵轴: 命中率越高, 平均访问时间越接近上层存储的访问时间 T_1 ; 命中率越低, 平均访问时间越接近 $T_1 + T_2$, 整体呈随命中率提升而下降的趋势。

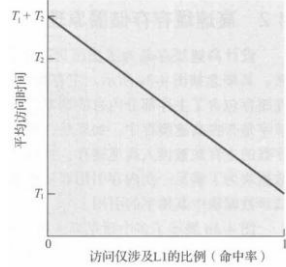


图 4-2 仅涉及 L1 访问的性能 (命中率)

5. Access Methods 四种存储器访问方法

访问方式	核心规则	访问时间特性	代表介质
顺序访问 (Sequential)	从存储起始位置开始顺序读取, 必须经过中间所有数据	访问时间取决于数据位置, 与上一次访问位置相关	磁带
直接访问 (Direct)	每个数据块有唯一地址, 先跳转到块所在区域, 再顺序查找定位	访问时间与数据位置相关	磁盘
随机访问 (Random)	每个地址有唯一物理定位, 可直接访问任意地址	访问时间固定, 与之前的访问位置无关	RAM (主存)
相联访问 (Associative)	通过对比数据内容特征定位, 无需指定地址	访问时间固定, 与位置无关	Cache

6. Access time, Cycle time, Transfer rate 访问时间、周期、传输速率定义

访问时间 (延迟): 对于随机访问存储器, 是执行读或写操作所需的时间。对于非随机访问存储器, 是将读写机制定位到所需位置所需的时间。

存储器周期时间: 主要针对随机访问存储器, 它等于访问时间加上第二次访问开始前系统所需的任何额外时间。

传输速率: 数据传入或传出存储器的速率。对于非随机访问存储器, 计算读或写 n 位数据的平均时间 T_n 的公式为:

$$T_n = T_1(A) + n/R \quad (\text{其中 } T_1(A) \text{ 为平均访问时间, } n \text{ 为位数, } R \text{ 为传输速率})$$

7. Cache principle 缓存的原理 (重点)

Cache 是弥合 CPU 与主存速度差距的核心部件, 核心原理如下:

- 定位: 介于 CPU 与主存之间的高速小容量半导体存储器, 可集成在 CPU 片内或作为独立模块, 存储主存数据的副本。

- 理论依据: 局部性原理, 程序运行时内存访问集中在局部区域。

- 核心逻辑: CPU 读取数据时优先访问 Cache:

- **命中**: 数据在 Cache 中, 直接高速读取, 无需访问主存;
- **缺失**: 数据不在 Cache 中, 先将主存中包含该数据的完整数据块加载到 Cache, 再将数据传给 CPU。

- 架构演进: 现代处理器普遍采用多级 Cache ($L1 \rightarrow L2 \rightarrow L3$), 逐级容量增大、速度降低, 兼顾访问速度与命中率。

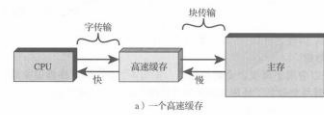


图 4-3 高速缓存和主存

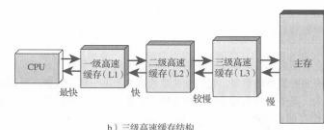


图 4-3 三级高速缓存结构

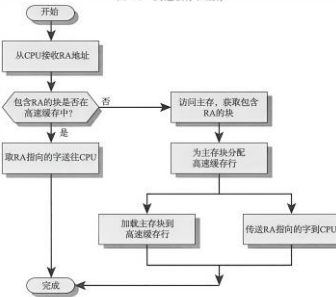


图 4-5 高速缓存逻辑操作

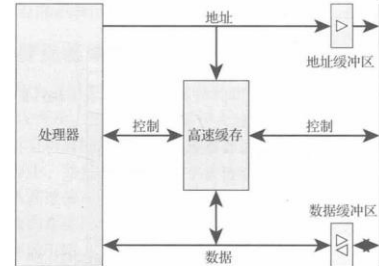


图 4-6 高速缓存典型结构

8. 逻辑缓存和物理缓存 处理器访问 Cache 时引用主存地址

按照访问使用的地址类型划分:

- **逻辑缓存 (Logical Cache)**: 直接使用 CPU 发出的逻辑地址

(虚拟地址) 访问缓存。

- 特点: 访问速度更快, 无需等待地址转换; 但进程上下文切换时需要清空整个缓存, 或增加 PID 位区分不同进程的地址空间。

原理: Hit 时, CPU 从逻辑地址 cache 中取走数据, 无需 MMU 将逻辑地址转化为物理地址; Miss 时, 逻辑地址转化为物理地址 (经过 MMU), 至 Memory 取数据块到 Cache 中

- **物理缓存 (Physical Cache)**: 使用 MMU 转换后的物理地址访问缓存。

- 特点: 上下文切换无需刷新缓存, 地址空间无歧义; 但需要先完成地址转换才能访问缓存, 访问延迟略高。

原理: Hit 与 Miss 时, 逻辑地址均需转化为物理地址; Miss 时, 到 Memory 中取数据至 Cache 中。Hit 时, 不用到 Memory 中取数据至 Cache

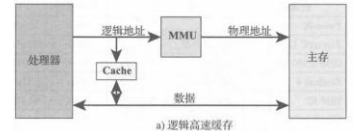
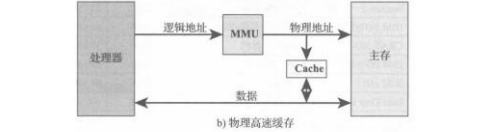


图 4-7 逻辑和物理高速缓存



高速缓存大小: Cache 由 SRAM 构成, 速度快, 单位成本高

9. Cache line and memory block 缓存行与内存块的定义

- **内存块 (Memory Block)**: 主存的最小传输单元, 由连续的多个字/字节组成, 是 Cache 与主存之间数据交换的基本单位。

- **缓存行 (Cache Line)**: Cache 的最小存储单元, 每行用于存放一个完整的存储块; 每行除数据区外, 还包含标签 (Tag)、有效位等控制信息。Cache 的总行数远少于主存的总块数。

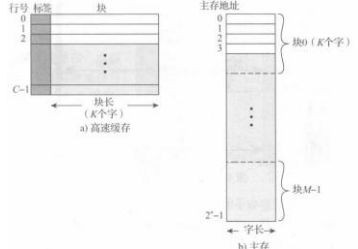


图 4-4 高速缓存 / 主存结构

10. Cache Tag and its role 缓存的 Tag 位及其作用

- 定义: **Tag 位**是主存地址的高位部分, 会随主存块一同存储在对应的 Cache 行中。

- 核心作用: 唯一标识当前 Cache 行存放的是哪一个主存块。访问 Cache 时, 通过将 CPU 地址中的 Tag 字段与 Cache 行存储的 Tag 进行对比, 结合有效位即可判断目标主存块是否存在于 Cache 中, 完成命中/缺失的判断。

11. Mapping Function 三种缓存的映射方法 (重点)

映射规则决定主存块与 Cache 行的对应关系, 共三类:

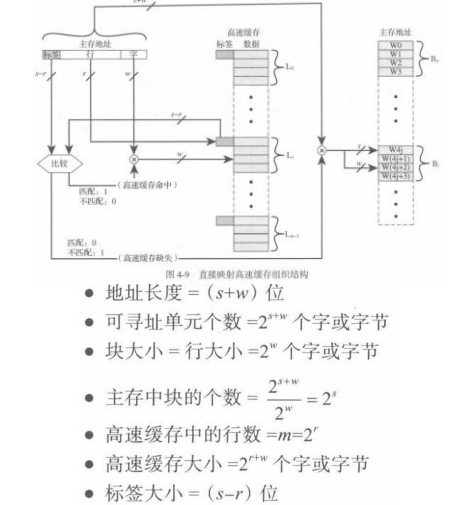
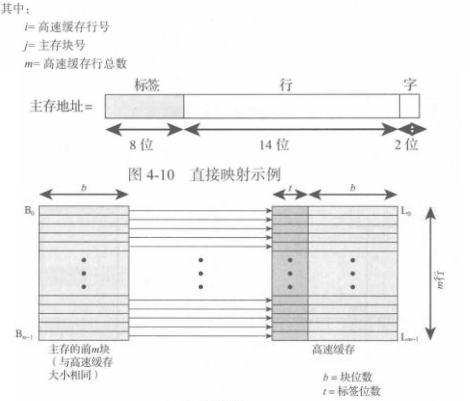
(1) **直接映射 (Direct Mapping)**

- 规则: 每个主存块只能映射到 Cache 中唯一的固定行, 映射公式: $i = j \bmod m$ (i 为 Cache 行号, j 为主存块号, m 为 Cache 总行数)。

- 地址结构: Tag 标签 + Line 行号 + Word 字内偏移

- 优缺点: 实现简单、硬件成本低; 但冲突类失率高, 多个频繁访问的块映射到同一行时会出现“抖动”。

通过中间 14 位 bit 找到 Cache 中行数，通过高 8 位 bit 与 Cache 中的 Tag 比较，s(s-r+r)块外寻址，w 块内寻址，s-r 定位主存，主存位数=微处理器位数=地址长度，主存大小=2^{s+w}



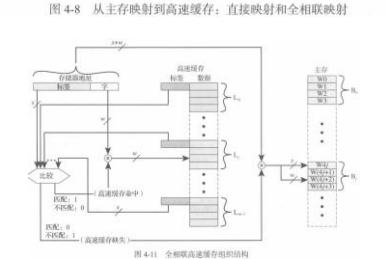
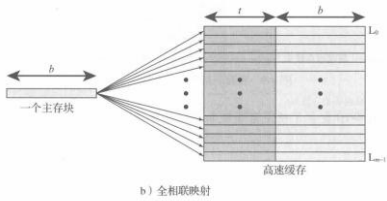
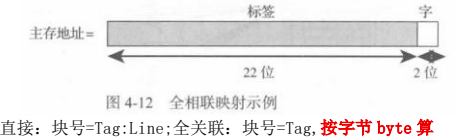
和地址一起存储在缓存中的其他地址有 2^w 个（轮换字内容）

(2) 全相联映射 (Associative Mapping)

规则：主存块可以加载到 Cache 的任意一行，无固定位置约束。

地址结构：Tag 标签 + Word 字内偏移，需要并行比较所有 Cache 行的 Tag 判断命中。

优缺点：冲突缺失率最低，缓存空间利用率最高；但查找电路复杂、硬件成本高，仅适用于小容量 Cache，花时间。比较高 22 位 bit 与 Cache 中 Tag



- 地址长度 = (s+w) 位
- 可寻址单元个数 = 2^{s+w} 个字或字节
- 块大小 = 行大小 = 2^r 个字或字节
- 主存中块的个数 = $\frac{2^{s+w}}{2^r} = 2^s$
- 高速缓存中的行数为未定
- 标签大小 = s 位

(3) 组相联映射 (Set Associative Mapping)

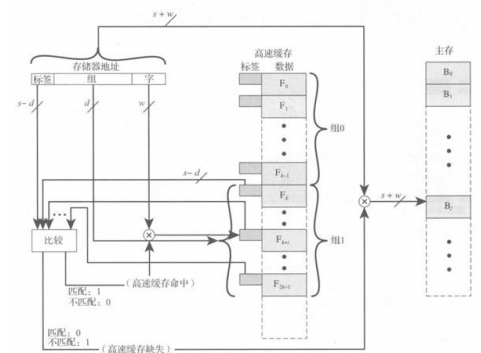
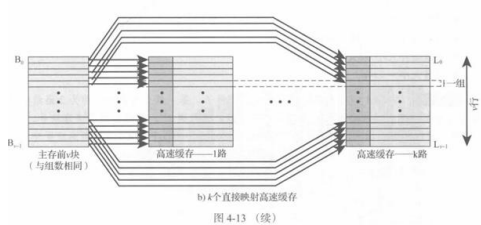
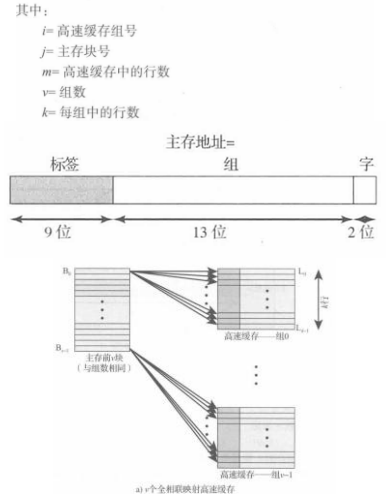
规则：前两种方案的折中。Cache 划分为多个组，每组包含 k 行；主存块映射到唯一的组，可放在组内任意一行。映射公式： $i = j \bmod v$ (i 为组号，v 为总组数)，总行数 $m = v \times k$ 。

地址结构：Tag 标签 + Set 组号 + Word 字内偏移

演化关系：k=1 时退化为直接映射；v=1 时退化为全相联映射。

工业界主流为 2 路、4 路组相联。k=way 路数

$m = v \times k$
 $i = j \bmod v$



- 地址长度 = (s+w) 位
- 可寻址单元个数 = 2^{s+w} 个字或字节
- 块大小 = 行大小 = 2^r 个字或字节
- 主存中块的个数 = $\frac{2^{s+w}}{2^r} = 2^s$
- 组中行数为 k

核心作用：当 Cache 访问命中后，在 Cache 行（完整主存块）的内部，精准定位到目标字节/字，完成数据的精准读取。

14. How to access a cache-memory system 访问缓存的步骤（重点）

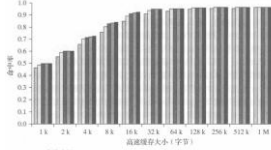
完整读取流程分为 4 步：结合具体地址

- 地址解析 (Address resolution)：将 CPU 发出的主存地址，按照映射规则拆分为 Tag 标签、行/组号、字内偏移三个部分。
- 定位位置 (Locate set/line)：根据地址中的行号/组号，找到对应的 Cache 行（直接映射）或 Cache 组（组相联映射）。
- 标记对比 (Compare tag)：将地址中的 Tag 字段，与选中行/组内所有行存储的 Tag 对比，同时检查有效位，判断是否命中。
- 数据读取 (Read byte)：

- 命中：若（有一路）Tag 匹配，根据字内偏移从对应 Cache 行中取出目标数据，传给 CPU，访问完成。
- 缺失：若（所有路）均不匹配，先访问主存读取完整的目标主存块，根据替换策略，将块加载到 Cache 对应位置并更新 Tag，最后将目标数据传给 CPU。

15. Diagram for cache access 缓存访问步骤的图形表示 三类相关图示：

- Cache 读取流程图：以「接收 CPU 地址」为起点，经过「是否命中」的判断分支，命中则直接取数，缺失则访问主存、加载 Cache、再取数，最终结束。
- Cache 硬件组织架构图：展示地址缓冲、处理器、Cache 控制器、数据缓冲、主存之间的连接与控制关系。
- 三类映射组织结构图：分别直观展示直接映射、全相联映射、组相联映射下，地址拆分、行/组定位、Tag 对比、数据读取的硬件逻辑。



- 64KB 以下直接和组相联差别显著 适中缓存大小
- 2.4KB 时 2 路与 4 路差距最小 合适相联度
- Cache 复杂度和关联度成正比，超过 32KB 后不会有显著性能提高

16. Write Policy 写策略

准则：除非主存中的数据是最新的，否则绝不允许覆盖（替换）一个缓存行；cache 是主存的副本，CPU 对 cache 更改，主存须同步修改

核心目标是保证 Cache 与主存的数据一致性，共两类核心方案：

- 通写 (Write through)
- 规则：每次写操作同时更新 Cache 和主存，两者数据实时一致。

- 所有写操作同时写入主存和缓存
- 多个 CPU 监控主存流量，保持本地缓存最新状态
- 特点：一致性好、实现简单；但所有写操作都需要访问主存，写流量大、写操作速度慢。循环次数*时间，修改内容在不同行

- 写回 (Write back)
- 规则：写操作仅更新 Cache 数据，同时为该行设置脏位 (Dirty bit) 标记修改；只有当该行被替换出 Cache 时，若脏位为 1，才将整行写回主存。

- 旧块没被写过，新块直接覆盖旧块
- 旧块被执行了写操作，需要写回主存后，再被新块覆盖
- 写回策略核心：CPU 只更新自己本地缓存，而不立即将数据写回主存 缓存行大小*时间 修改内容在同一行
- 特点：写操作无需访问主存，写流量小、效率高；但存在 Cache 与主存数据不一致的问题，I/O 访问主存需经过 Cache。

- (3) 适用场景
- 写直达：适合对一致性要求高、写操作频率低的场景。
 - 写回：适合写操作频繁、数据块被替换前会被多次写入的场景，能大幅降低写访问次数。

17. Cache coherency 缓存的一致性

解决多 CPU 系统中，多个私有 Cache 之间、Cache 与主存之间的**数据不一致**问题，三类方案：

- 总线监听+写直达（Bus watching with write through）：所有 Cache 控制器监听总线上的主存写操作；若发现主存块被其他 CPU 修改，就将本地 Cache 中对应的块标记为无效。
- 硬件透明性（Hardware transparency）：所有主存更新都通过 Cache 硬件自动同步到所有 CPU 的 Cache 副本，对上层软件透明。
- 不可高速缓存存储器（Noncacheable memory）：将多 CPU 共享的内存区域标记为不可缓存，所有访问直接操作主存，从根源上避免一致性问题。

数据行大小：由局部性原理，程序在访问某个内存地址时，通常很快访问其附近地址

1. 当块较小时，每次出块仅加载很少的数据。增加块大小意味着每次出块会将目标数据及周围更多数据一次性加载至 cache 中，因此块大小增加的初期，命中率会上升
2. 当块大小持续变大，缓存内的块总数会急剧减少，意味着多个内存地址映射到同一位置的概率升高；缓存行容易出现缺失，即新数据块会不断把旧块替换出去，即便旧块的数据后续仍会被访问

18. Replacement Algorithms 替换方法

Cache 已满、发生缺失需要加载新块时，选择**被替换行的算法**：

- 直接映射场景：每个主存块只有唯一对对应行，无选择空间，直接替换该行即可。
- 全相联/组相联场景，四类主流算法：
 1. **最近最少使用**（LRU）：替换过去最久没有被访问的块，性能最优，是最常用的替换算法。
 2. **先进先出**（FIFO）：替换在 Cache 中驻留时间最长、最早加载的块，实现简单但不符合局部性规律。
 3. **最不经常使用**（LFU）：替换累计被访问次数最少的块，对近期热点不敏感。
 4. **随机**（Random）：随机选择块替换，实现最简单，性能略低于基于使用统计的算法。

Ch14 Processor Structure and Function 知识点总结

14.1 处理器的组织

为了理解处理器的组成，让我们来考虑一下对处理器的需求，也就是它必须要做的工作：

- **读取指令** 处理器从存储器（寄存器、高速缓存、主存）读取指令。
 - **解释指令** 对指令进行译码，以决定需要怎样的操作。
 - **读取数据** 执行指令可能需要从存储器或 I/O 模块读取数据。
 - **处理数据** 执行指令可能需要对数据进行一些算术或逻辑操作。
 - **写数据** 执行的结果可能需要把数据写回存储器或 I/O 模块。
- 要完成上述这些工作，就要明白处理器需要暂时存储一些数据。处理器必须记住最后一条指令的位置，这样它才知道到哪里去取下一条指令。它还需要在指令执行的时候暂时保存指令和数据。换句话说，处理器需要一个小型的内部存储器。
- 图 14-1 是处理器的简化视图，显示了它通过系统总线与系统其他部分的连接。第 3 章描述的任何互连结构都需要类似的接口。读者会记得处理器的

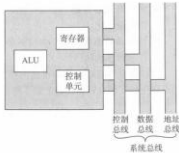


图 14-1 带系统总线的 CPU

4.22 某计算机有高速缓存，主存和用于虚拟存储器的磁盘。如果一个被引用的字在高速缓存中，那么访问它需要 20 ns。如果这个字在主存中而在高速缓存中，那么需要 60 ns 将其加载到高速缓存，然后再次启动对它的访问。如果这个字不在主存中，则需要 12 ms 把它从硬盘中取出，之后用 60 ns 复制到高速缓存，再重启对它的访问。高速缓存命中率为 0.9，主存命中率为 0.6，那么在这个系统上访问一个被引用字

- The average time
$$= T_1 + (1 - H_1) [T_2 + (1 - H_2) T_3]$$
$$= 20 + (1 - 0.9) * [60 + (1 - 0.6) * 12,000,000]$$
$$= 480026ns$$
- The average time
$$= 20 * 0.9 + 80 * 0.06 + 12,000,080ns * 0.04$$
$$= 18 + 4.8 + 480003.2 = 480026ns$$

主要组件是算术逻辑单元（ALU）和控制单元（CU）。ALU 执行实际的计算或是数据处理。控制单元控制数据和指令进入处理器的移动，以及 ALU 的操作。此外，该图还展示了最小的内部存储器。它由一组被称为寄存器存储位置构成。

图 14-2 展示了略微详细的处理器。该图显示了数据传输和逻辑控制路径，包括了标注为“内部处理器总线”的部分。这个部分用于在不同的寄存器和 ALU 之间传递数据，因为 ALU 实际上只能操作处理器内部存储器中的数据。图中还显示了 ALU 的典型基本要素。请注意计算机整体的内部结构与处理器内部结构之间的相似性。在这两种情况下，都有一部分的主要部件（计算机：处理器、I/O、内存；处理器：控制单元、ALU、寄存器）通过数据路径来连接。

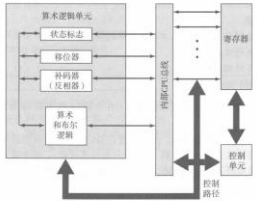


图 14-2 CPU 的内部结构

1. User-visible registers 定义 + General purpose registers 详解

(1) **User-visible registers**（用户可见寄存器）定义

用户可见寄存器是 CPU 寄存器层次中可被机器语言直接引用的寄存器，是存储器层次结构的最顶层，作为 CPU 的临时高速存储，用于减少对主存的访问，提升执行效率。

分类：**通用目的寄存器、数据寄存器、地址寄存器、条件码（标志）寄存器**。

(2) **General purpose registers**（通用寄存器）详解

通用寄存器可由程序员分配给多种功能，分为两类设计：

- 真正通用设计：寄存器功能与操作正交，可用于任意数据处理、地址计算等场景；优点是灵活性高、程序员可选方案多；缺点是指令长度更长、译码复杂度更高。
 - 受限专用设计：寄存器被分配特定功能，例如：
 - 浮点寄存器：专用于浮点运算；
 - EBX：栈基址寄存器；ECX：循环计数器；
 - EAX、EDX：数据寄存器，专用于算术运算；
 - 段指针、索引寄存器、栈指针：专用于地址计算与栈操作。
- 优点：指令更短、执行速度更快；缺点是灵活性低。

数量与大小：通用寄存器数量通常为 8~32 个，数量过少会增加内存访问次数，过多不会显著减少内存引用；RISC 架构采用上百个寄存器是新的设计方向。寄存器大小需足够容纳完整地址与完整字，常支持两个寄存器组合存储长整型数据。

(3) **数据寄存器**（Data Registers）

与地址寄存器可能部分或完全分离。

功能：仅用于保存数据，不能用于计算操作数的地址。

(4) **地址寄存器**（Address Registers）

可以是通用的，也可以专用于特定的寻址模式。常见类型包括：**段指针**（Segment Pointers）：用于分段寻址机制中，存放段的基地址（例如，操作系统使用一个，当前进程使用另一个）。**变址寄存器**（Index Registers）：用于变址寻址，有时也用于自动变址。

栈指针（Stack Pointers）：如果存在用户可见的栈寻址，专门用于指示栈顶。这允许 push、pop 等栈指令进行隐含寻址，无需在指令中显式包含栈操作数。

(5)**条件码**（Condition Codes / 标志）**寄存器**

条件码是由处理器硬件根据算术或逻辑操作的结果（如正、负、零、溢出）设置的位。它们构成了控制寄存器的一部分。

访问权限：对用户至少是部分可见的。机器指令通常允许通过隐式引用来读取这些位（用于条件分支测试），但**程序员通常不能直接改变它们**。

架构差异：并非所有处理器都使用条件码。例如，基于 IA-64 体系结构的处理器和 MIPS 处理器就不使用条件码，而是通过条件分支指令指定比较操作，并直接根据比较结果执行分支。

(6) 寄存器设计的三个核心问题

完全通用 vs. 专用化：

专用寄存器： 可以将操作数说明符所指的寄存器类型隐含在操作码中，从而节省指令的位数。缺点是限制了程序员的灵活性。

通用寄存器： 灵活性高，但可能需要更多位来标识具体的寄存器。

寄存器的数量：

数量会影响指令集设计，更多的寄存器需要更多的操作数说明符位数。

最佳范围： 通常在 8 ~ 32 个之间。寄存器过少会导致频繁访问内存；但寄存器过多也不会明显减少访存次数（注：某些 RISC 系统使用上百个寄存器属于例外情况）。

寄存器的长度：

地址寄存器： 必须足够长，以容纳系统最大的地址。

数据寄存器： 应能存放大多数数据类型的数据。有些机器允许拼接两个连续的寄存器来存放一个双倍长度的数值。

2. Control/status registers 定义及作用 + PC、MAR、MBR、IR 四个寄存器的作用 + PSW 详解

(1) **Control/status registers**（控制/状态寄存器）定义及作用

控制/状态寄存器用于控制处理器的运行操作，大部分对用户不可见，仅在控制模式或操作系统模式下可被机器指令访问。核心作用是管理指令执行流程、记录处理器运行状态、支持中断与特权级控制。

(2) 四个核心寄存器的作用

- **Program Counter**（PC，程序计数器）：始终存储下一条要执行的指令的地址，每取完一条指令后自动自增，控制指令执行的顺序。
 - **Instruction Register**（IR，指令寄存器）：存放当前正在执行的指令，用于控制单元对指令进行解码和分析，是指令执行的临时存储。
 - **Memory Address Register**（MAR，内存地址寄存器）：连接系统地址总线，存放要访问的内存地址，所有内存读写操作的地址都通过 MAR 传递。
 - **Memory Buffer Register**（MBR，内存缓冲寄存器）：连接系统数据总线，存放从内存读取的数据或要写入内存的数据，是 CPU 与内存之间的数据交换缓冲。
- (3) **PSW**（Program Status Word，程序状态字）详解
- PSW 是一组状态位的集合，包含：
- 条件码（标志位）：记录上次运算结果的状态，包括结果符号位、零标志（结果为 0）、进位标志、相等标志、溢出标志；
 - 中断使能/禁用位：控制是否响应外部中断；
 - 监督模式位（内核模式/特权模式位）：标记当前处理器运行级别，监督模式下可执行特权指令，供操作系统使用，对用户程序不可用。

许多处理器设计都有包含状态信息的寄存器或寄存器组，通常称为程序状态字（PSW）。PSW 一般含有条件码及其他状态信息。常见字段或标志如下：

- 符号 包含最近算术运算结果的符号位。
- 零 当结果为 0 时置位。
- 进位 如果运算结果有向更高数位（加法）或借位（减法）则置位。用于多字节算术运算。
- 相等 如果逻辑比较结果为相等，则置位。
- 溢出 用于表示算术溢出。
- 中断允许/禁止 用于允许或禁止中断。



- 管理员 指示处理器是在管理员模式还是用户模式下执行。某些特权指令只能在管理员模式下执行。某些内存区域只能在管理员模式下访问。

3. Instruction cycle 指令循环

指令周期是 CPU 执行一条指令的完整过程，是处理器运行的基本周期，整体为循环结构：处理器重复执行「取指→（间址）→执行→（中断）」的流程，直到程序结束。

指令周期的核心是将指令执行分解为多个子阶段，通过阶段重叠（流水线）提升执行效率；不同指令的周期长度不同，简单指令周期短，复杂指令（含间址、中断）周期长。

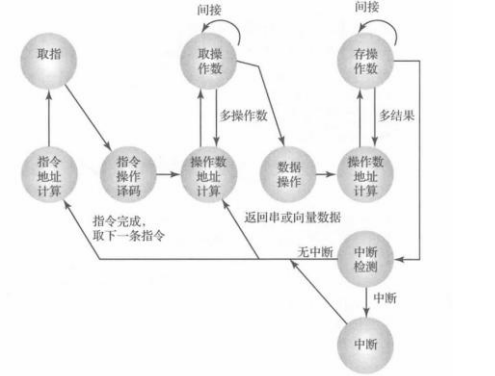


图 14-5 指令周期状态图

4. Stages：Fetch/Indirect/Execute/Interrupt 指令常规的几个步骤

完整指令周期包含四个可选子阶段：

- (1) **Fetch（取指阶段）**
流程：PC 中的指令地址送入 MAR，放到地址总线；控制单元发起内存读请求，指令从内存读取到 MBR，再复制到 IR；同时 PC 自增，指向下一条指令。
作用：从内存获取下一条要执行的指令。
- (2) **Indirect（间址阶段）**

触发条件：指令使用间接寻址时执行。
流程：检查 IR 中的地址字段，将 MBR 的地址部分送入 MAR，发起内存读，获取操作数的真实地址，更新 MBR；可多次执行以支持多级间接寻址。

作用：解析间接寻址，获取操作数的真实内存地址。

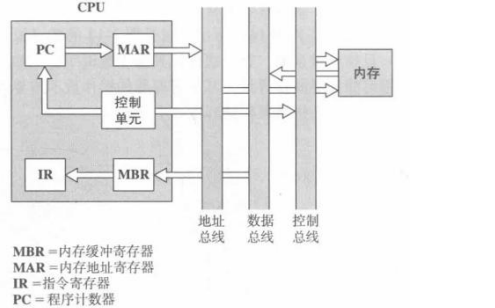


图 14-6 取指周期的数据流

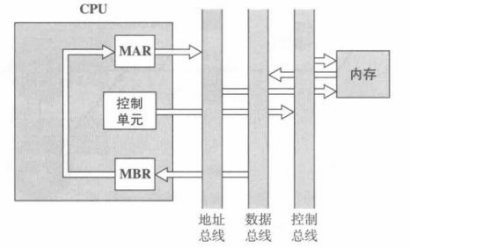


图 14-7 间接周期的数据流

(3) **Execute（执行阶段）**
流程：根据 IR 中的指令操作码，执行对应操作，包括内存读写、

I/O 操作、寄存器数据传输、ALU 运算等。
作用：完成指令的核心功能，是指令周期的核心阶段。
(4) **Interrupt (中断阶段)**
触发条件：执行阶段结束后检查到有未处理的中断时执行。
流程：将当前 PC 值（断点）保存到栈中，PC 加载中断处理程序的入口地址，准备执行中断服务程序；中断处理完成后恢复断点，继续执行原程序。
作用：响应外部中断，支持多任务、异常处理与外设交互。

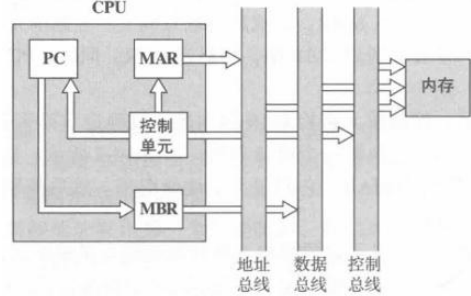


图 14-8 中断周期的数据流

5. Necessary stages: 指令必不可少的两个步骤 Fetch, Execute
任何指令的执行都必须包含**两个核心阶段**，是指令周期的必要组成部分：

- (1) **Fetch (取指阶段)**：所有指令都必须先从内存中读取到 CPU，否则无法执行，是指令执行的前提。
 - (2) **Execute (执行阶段)**：所有指令都需要完成对应的操作功能，是指令的核心目的。
- 间址阶段仅在使用间接寻址的指令中存在，中断阶段仅在有中
断请求时存在，都不是所有指令的必需阶段；仅取指和执行是
每条指令必不可少的步骤。

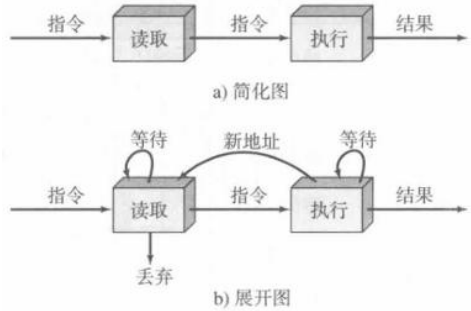


图 14-9 两阶段的指令流水线

6. Pipeline principle 流水线原理 (重点)
指令流水线是类似工业装配线的并行执行技术，核心原理是将指令执行分解为多个独立的子阶段，每个阶段由专门的硬件单元处理，多个指令的不同阶段可同时重叠执行，大幅提升 CPU 的指令吞吐量。
标准 6 级流水线分解：
1. FI (Fetch Instruction)：取指令
2. DI (Decode Instruction)：指令译码
3. CO (Calculate Operands)：计算操作数地址
4. FO (Fetch Operands)：取操作数
5. EI (Execute Instruction)：执行指令
6. WO (Write Operand)：写回操作数

核心优势：单条指令的执行延迟不变，但单位时间内完成的指令数（吞吐量）大幅提升；流水线深度越大，理论吞吐量越高。
7. Pipeline diagram 流水线图形 (包括无 branch 和有 branch 的流水线) (重点)
(1) **无分支 (无跳转) 的流水线时序图**
理想状态下，每个时钟周期启动一条新指令，各阶段完全重叠：
- 第 1 周期：指令 1 取指 (FI)
- 第 2 周期：指令 1 译码 (DI)，指令 2 取指 (FI)
- 第 3 周期：指令 1 算址 (CO)，指令 2 译码 (DI)，指令 3 取指 (FI)
以此类推，n 条指令的总执行时间为 $[k + (n-1)] \tau$ ，k 为流水线级数， τ 为单周期时间，所有阶段无停顿。
(2) **有分支 (条件跳转) 的流水线时序图**
当执行到分支指令时，分支的目标地址需要在执行阶段才能确定，此时流水线中已经预取了后续的多条指令；若分支跳转发生，预取的指令全部作废，流水线出现「气泡」(停顿)，产生分支惩罚 (Branch Penalty)：
- 分支指令执行到 EI 阶段时才确认跳转，此时流水线中已预取了后续 3-4 条指令；
- 跳转发生时，作废已预取的指令，清空流水线，重新从分支目标地址取指；
- 总执行时间大幅增加，分支惩罚等于流水线中已预取的指令数。

分支：有 WO 最后重合，无 WO 最后不重合

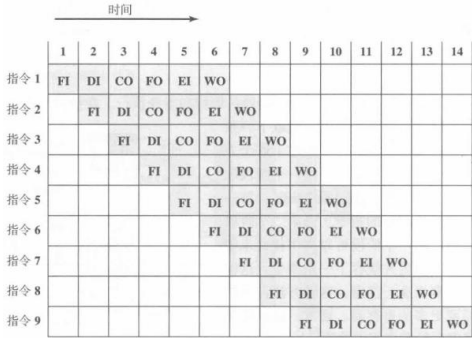


图 14-10 指令流水线操作的时序图

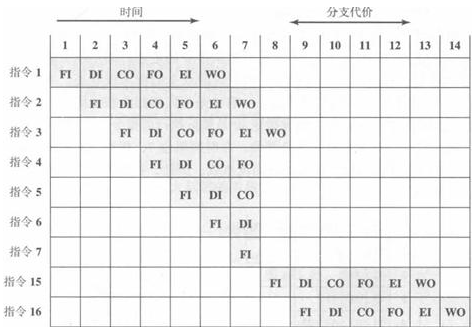


图 14-11 指令流水线操作中条件分支的影响

指令 3 分支到指令 15 (3EI 后接 15FI, WO 不影响)

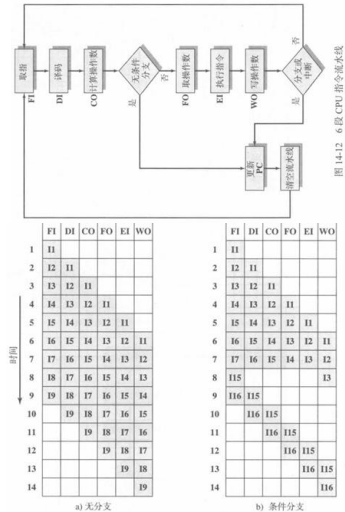


图 14-13 另一种流水线描述



不等长阶段流水线

8. Speed up factor (公式、图形、无 branch 和有 branch 情况的计算，不等长 stage 情况的计算) 重点

(1) **理想等长流水线加速比公式**

k 级流水线 (k 个阶段)，执行 n 条指令，单级时间 τ ：

- 非流水线总时间： $T_0 = n \times k \times \tau$

- 流水线总时间： $T_k = [k + (n-1)] \times \tau$

- 加速比： $S_k = T_0 / T_k = (n k) / [k + n - 1]$

当 $n \rightarrow \infty$ 时， $S_k \rightarrow k$ ，理论最大加速比等于流水线级数。

(2) **加速比图形特征**

- 横轴为指令数 n，纵轴为加速比 S_k ：流水线级数 k 越大，加速比随 n 增长越快，极限值越高；

- 横轴为流水线级数 k，纵轴为加速比 S_k ：指令数 n 越大，加速比随 k 增长越快，小 n 时流水线收益不明显。

(3) **有分支的流水线加速比 见第 10 页**

设分支概率为 p，分支惩罚为 b (停顿周期数)：

平均每条指令周期数 $CPI = 1 + p \times b$

加速比 $S = k / CPI = k / (1 + p b)$

分支概率越高、惩罚越大，加速比下降越明显。

(4) **不等长 stage 的加速比计算**

流水线各阶段时间不同时，流水线周期由最长的阶段时间 τ_{max} 决定： $T_k = \sum \tau_i + (n-1) \tau_{max}$ —**一般**

总时间 $T_k = k \tau_{max}$ (其余周期强制拉长，**一般为 $\sum \tau_i$ 第一个指令时长**) + $(n-1) \tau_{max} = (k + n - 1) \tau_{max}$

非流水线总时间 $T_0 = n \times \sum \tau_i$ ($\sum \tau_i$ 为单条指令各阶段时间总和)

加速比 $S = T_0 / T_k = (n \sum \tau_i) / [(k + n - 1) \tau_{max}]$

此时最大加速比为 $\sum \tau_i / \tau_{max}$ ，由最慢阶段决定。

14.4.2 流水线性能

在本小节中，我们将开发一些简单的流水线性能度和相对加速比 (以 [HWAN93] 中的讨论为基础)。指令流水线的周期时间 τ 是通过流水线把一组指令推进一个阶段所需的时间；图 14-10 和图 14-11 中的每一列都代表一个周期时间。周期时间可以用如下表达式来决定：

$$\tau = \max_i [\tau_i] + d = \tau_e + d, 1 \leq i \leq k$$

其中：

τ_i = 流水线中第 i 段的电路延迟时间

τ_e = 最大流水段延迟 (通过经历最大延迟的阶段的延迟)

k = 指令流水线中的阶段数

d = 把信号和数据从一个阶段推进到下一个阶段所需的锁存器延迟

通常，延迟时间 d 等于一个时钟脉冲，且 $\tau_e \gg d$ 。现在假设要处理 n 条指令，没有分支。令 $T_{k,n}$ 为流水线执行这 n 条指令所需的总时间，流水线有 k 段。则

$$T_{k,n} = [k + (n-1)]\tau \quad (14.1)$$

完成第一条指令的执行总共需要 k 个周期，剩下的 (n-1) 条指令需要 (n-1) 个周期^①。这个等式很容易从图 14-10 得到验证。第 9 条指令在第 14 个时间周期完成：

$$14 = [6 + (9-1)]\tau$$

现在考虑一个功能相同但无流水线的处理器，假设其指令周期时间为 $k\tau_e$ 。与非流水线执行相比，指令流水线的加速比因子定义为：

$$S_k = \frac{T_{0,n}}{T_{k,n}} = \frac{n k \tau_e}{[k + (n-1)]\tau} = \frac{n k}{k + (n-1)} \quad (14.2)$$

图 14-14a 绘制了加速比因子，它是无分支时执行指令数的函数。可以预期，在极限情况下 ($n \rightarrow \infty$)，我们可以得到 k 倍的加速比。图 14-14b 显示的加速比因子是指令流水线中流水段数量的函数^②。在这种情况下，加速比因子接近于可以输入流水线的无分支指令的数量。因此，流水线的段数越多，加速的可能性就越大。然而，实际上，更多流水段的潜在收益会被成本的增加、段间的延迟和遇到分支后清洗流水线的需求所抵消。

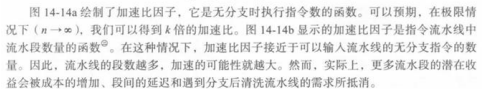


图 14-14 指令流水线的加速比因子

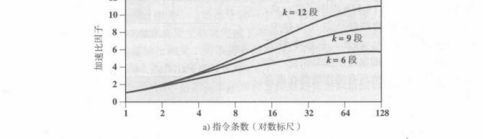


图 14-14 指令流水线的加速比因子

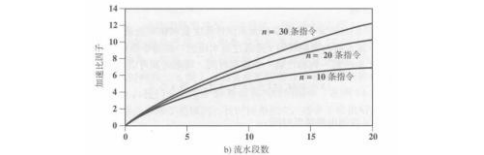


图 14-14 指令流水线的加速比因子

9. Stages with unequal time (流水线时间由最长的 stage 决定)

当流水线各阶段的执行时间不相等时，流水线的时钟周期由时间最长的阶段决定：

- 所有阶段必须同步工作，每个时钟周期所有阶段同时完成操作，因此快的阶段必须等待最慢的阶段完成；

- 最慢的阶段成为流水线的瓶颈，决定了整个流水线的最大吞吐率；

- 优化方向：拆分最长阶段为多个子阶段，平衡各阶段时间，提升流水线整体效率。

例：若流水线 5 个阶段时间分别为 100ps、80ps、150ps、100ps、90ps，则流水线周期为 150ps，所有阶段都按 150ps 周期工作。

10. 影响流水线的因素

流水线的**三类核心冒险** (Hazard) 会导致流水线停顿，降低效率：

(1) **资源冒险 (结构冒险)**

原因：硬件资源不足，多个阶段同时竞争同一硬件资源 (如单端口内存同时被取指和取数访问)；

解决：增加硬件资源 (如多端口内存、分离指令/数据 Cache)。

(2) **数据冒险**

原因：指令之间存在数据依赖，后一条指令需要前一条指令的运算结果，但结果还未写入；

分类：写后读 (RAW 真相关)、读后写 (WAR 反相关)、写后写 (WAW 输出相关)；

解决：数据转发 (旁路)、插入停顿、编译器指令调度。

(3) **控制冒险 (分支冒险)**

原因：分支指令、中断等改变指令流，预取的指令无效，产生流水线气泡；
解决：分支预测、预取分支目标、延迟分支、循环缓存等技术。
11. Dealing with branches: Multiple streams、Prefetch branch target、Loop buffer、Branch prediction、Delayed branching

分支冒险的五类解决方案：

(1) Multiple streams (多流水线)

原理：复制多套流水线硬件，同时预取分支的两个方向的指令，分支结果确定后选择正确的流水线继续执行；
优缺点：无需停顿，但硬件成本高，多分支时资源竞争严重，易导致冲突，需要流水线各阶段资源

(2) Prefetch branch target (预取分支目标)

原理：遇到分支指令时，除了预取顺序后续的指令，同时预取分支目标地址的指令，分支结果确定后直接使用已预取的目标指令，减少取指延迟；

特点：实现简单，IBM 360/91 采用该技术。

(3) Loop buffer (循环缓存)

原理：在取指阶段维护一个小容量高速缓存，保存最近 n 条取到的指令；遇到小循环或短跳转时，直接从缓存中取指令，无需访问内存；
优势：对小循环效率极高，完全消除循环内的取指延迟，适合循环密集型程序。

(4) Branch prediction (分支预测) 12

原理：根据分支的历史执行信息，预测分支是否跳转，按预测方向预取指令；
分类：静态预测（预测从不跳转，预测总是跳转，按操作码预测）、动态预测（跳转/不跳转开关，分支历史表）；预测错误时清空流水线回滚。

(5) Delayed branching (延迟分支) Ch15.13

原理：重新排列指令顺序，将分支指令后面的延迟槽填充与分支无关的有用指令，分支跳转时延迟槽的指令依然有效，不浪费周期，减少分支指令带来的控制冒险的影响
特点：无需额外硬件，由编译器完成指令调度，RISC 架构广泛采用。

12. Static branch prediction: When Predict taken? / When Predict not-taken?

静态分支预测基于固定规则，不依赖分支历史执行信息：

(1) Predict not-taken (预测不跳转)

适用场景：
- 所有分支默认预测不跳转，始终预取顺序下一条指令；
- 前向分支（跳转到地址更高的位置），通常为条件不满足时的错误处理分支，大概率不跳转；
优势：实现最简单，不会产生地址越界错误；缺点：向后循环分支预测错误率高。

(2) Predict taken (预测跳转)

适用场景：
- 所有分支默认预测跳转，始终预取分支目标地址的指令；
- 向后分支（跳转到地址更低的位置），通常为循环分支，大概率跳转；
优势：循环类分支预测准确率高；缺点：可能预取非法地址产生页错误，需要额外处理。

(3) 按操作码预测

根据指令类型预测：不同指令的跳转概率不同，例如循环指令、返回指令大概率跳转，条件判断指令根据类型判断；准确率可达 75% 以上。

13. Dynamic prediction (难点)：How to use history bit/bits / Error in prediction / Branch history table 作用

动态预测：

1. 跳转/不跳转开关

动态分支预测基于分支的历史执行记录，动态调整预测结果，准确率可达 80% 以上：

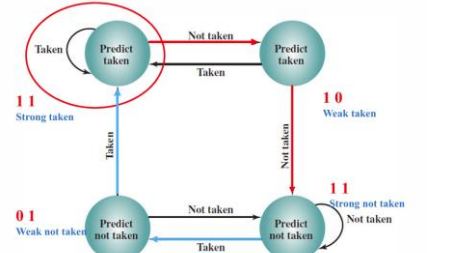
(1) 1 位历史位预测器 一次不同

- 原理：每个分支维护 1 个历史位，记录上次分支是否跳转；
- 历史位=1：预测跳转；分支实际跳转则保持 1，不跳转则改为 0；
- 历史位=0：预测不跳转；分支实际不跳转则保持 0，跳转则改为 1；
- 预测错误：循环执行时，第一次进入循环和最后一次退出循环会预测错误，n 次循环共 2 次错误。

(2) 2 位历史位预测器（饱和计数器，主流方案）两次不同

- 原理：每个分支维护 2 位状态，共 4 个状态：强跳转 (11)、弱跳转 (10)、弱不跳转 (01)、强不跳转 (00)；
- 跳转发生：状态向「跳转」方向移动 1 步，强跳转保持；
- 不跳转发生：状态向「不跳转」方向移动 1 步，强不跳转保持；
- 预测规则：状态高位为 1 则预测跳转，为 0 则预测不跳转；
- 预测错误：需要连续两次预测错误才会改变预测方向，抗干扰能力强，适合有波动的分支；
- n 次循环仅最后一次退出时可错，错误率远低于 1 位预测器。

Branch Prediction State Diagram



How many if it bit taken/not taken switch is adopted? Assume that the initial state of the switch is not taken.

1 bit: $B_2: N \rightarrow NT, B_1: 每次 B_2, N \rightarrow NT$

2 bit: $B_2: N \rightarrow NT, B_1: N \rightarrow NT, N \rightarrow NT, N \rightarrow NT$

Handwritten calculations: $2 \times 99 + 1 + 1 = 200$, $100 \times 1 + 1 = 101$

Code:
S=0;
for K=1 to 100 do
for M=1 to 20 do
S=S-M;
S=S-K;

2. Branch history table (分支历史表, BHT)

- 作用：存储所有分支的预测信息，是动态预测的硬件载体；
- 结构：表项包含分支指令地址、分支目标地址、预测状态位 (1/2 位)；
- 工作流程：
1. 取指时用分支地址查找 BHT，命中则按表中状态预测分支方向，预取对应地址指令；
2. 分支执行完成后，更新 BHT 中对应表项的预测状态；
3. 预测错误时，清空流水线，回滚到正确地址重新取指，更新 BHT。
- 优势：容量大，可记录大量分支的历史信息，支持高精度动态预测。

10	11	12	13	14	15	16 进制	
A	B	C	D	E	F		
128	64	32	16	8	4	2	1
1	1	1	1	1	1	1	1
EAX(4H,32 位)							
数字转 ASCII:AND AL,30H				AX(2H)			
				AH(1H)		AL(1H)	

数字(0-9)ASCII 码: 48 (30H) ~57 (39H) ASCII 到 BCD: AND AL,0FH
大写字母: 65 (41H) ~90 (5AH); 小写字母: 97 (61H) ~122 (7AH) 字母前加 0
 $2^{10} = 1KB$; $2^{16} = 64KB = 65535$; $2^{20} = 1MB$; $2^{24} = 16MB$; $2^{32} = 4GB$
DB: 1 字节; DW: 2 字节; DD: 4 字节; DQ: 8 字节; DT: 10 字节
1 个字节 = 8 位(二进制)=2 位(十六进制) 00H 转 BCD: 大于 9 每位+6H

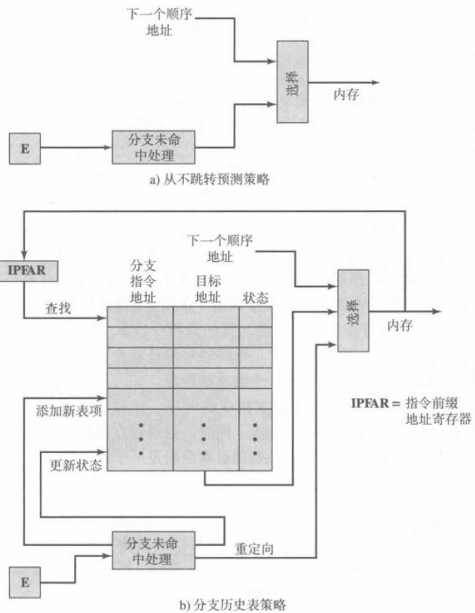


图 14-20 分支处理

Ch15 Reduced Instruction Set Computers 知识点总结

1. Major advances since the birth of the computer (简介)

计算机诞生以来的主要架构进步 (里程碑)：

- (1) 系列机概念 (The family concept)
1964 年 IBM System/360、DEC PDP-8 首次实现，将架构与实现分离，同一架构可兼容不同性能的硬件实现，是现代计算机兼容体系的基础。
- (2) 微程序控制单元 (Microprogrammed controlunit)
1951 年 Wilkes 提出，1964 年 IBM S/360 实现，用微码实现复杂指令的控制逻辑，降低了硬件设计复杂度，是 CISC 架构的核心技术基础。
- (3) 高速缓存 (Cache memory)
1968 年 IBM S/360 Model 85 引入，弥合 CPU 与主存的速度差，是存储器层次结构的核心组件。
- (4) 流水线 (Pipelining)
将取指-执行周期并行化，引入指令级并行，大幅提升 CPU 指令吞吐量。
- (5) 多处理器 (Multiple processors)
多 CPU 并行执行，引入线程级与任务级并行。

(6) 下一步架构进步：RISC (精简指令集计算机)，是架构发展的下一个核心里程碑。
高级语言 (HLL) 的特点包括：1) 允许程序员更精确地表示算法；2) 允许编译器处理在程序员算法表达式中不重要的细节；3) 通常自然地支持结构化编程或面向对象设计的使用。

语义差距：HLL 中提供的操作与计算机体系结构中提供的操作之间的差异。这种差距表现在以下方面：执行效率低、机器程序过大和编译器复杂。

寻找另一种不同的方法：使支持 HLL 的体系结构更简单，而不是更复杂。
指令执行的特点：1. 执行的操作：决定了处理器要执行的功能以及它与内存的交互。2. 使用的操作数：操作数的类型及其使用频率决定了存储它们的内存组织以及访问它们的寻址模式。3. 执行序列：决定了控制与流水线结构

2. RISC and its characteristics (注意区分 RISC 和 RISA 两个概念)

RISC (Reduced Instruction Set Computer 精简指令集计算机) 是采用 RISA 架构的计算机核心特征共 3 个：

- (1) 大量通用寄存器，或使用编译器技术优化寄存器使用；
- (2) 有限且简单的指令集；
- (3) 重点优化指令流水线。

原理：1. 让硬件更简单，由编译器负责将 HLL 的指令翻译为简单指令组合 2. 简化硬件，带来高效率，低功耗和更大存储器空间，从而实现软件加速 3. 编译器承担更多责任，硬件专注于执行大量简单快速操作

CISC (复杂指令集计算机 Complex Instruction Set Computer) 是一种采用复杂指令设计的计算机体系结构。该架构通过硬件实现常用功能降低编程难度，处理器电路复杂度较高，包含丰富的微指令集，允许单个指令周期处理多步低阶操作，一条指令=多条 RISC 指令使用原因：1. 编译器简化 2. 预期 CISC 将产生更小、更快的程序 3. 相较于 RISC，使用 CISC 只减少了很少的代码量，或者没有减少

RISC 与 CISC 的核心差异：

- 指令数量：RISC 仅 69-225 条，CISC 为 208-303 条；
- 指令长度：RISC 固定 4 字节，CISC 为 1-57 字节可变；
- 寻址方式：RISC 仅 1-2 种，CISC 为 4-22 种；
- 通用寄存器数量：RISC 为 32-520 个，CISC 仅 8-16 个。

注意：RISC 是计算机硬件实现，RISA 是其底层指令集架构，二者是实现与架构的对应关系，指令集简单是 RISC 核心特征之一。

3. RISA and its characteristics

RISA (Reduced Instruction Set Architecture, 精简指令集架构) 是指令集架构层面的定义，核心特征共 4 个：

- (1) 单周期执行一条指令：所有指令的执行时间均为一个时钟周期，简化流水线设计；
- (2) 仅支持寄存器到寄存器的运算操作：采用 load/store 架构，只有 load 和 store 指令可以访问内存，所有运算操作均在寄存器间完成，减少内存访问延迟；
- (3) 极少、简单的寻址方式：通常仅支持 1-2 种寻址方式，简化地址译码逻辑；
- (4) 极少、简单的指令格式：指令长度固定，格式统一，简化指令译码与流水线控制。

RISA 是 RISC 计算机的底层指令架构，RISC 是采用 RISA 架构的硬件实现，二者是架构与实现的关系。

4. Motivation of developing RISC (简介，根据高级语言运行特点的分析得到的结论)

RISC 的开动机完全基于高级语言 (HLL) 程序的动态执行特征分析，核心结论如下：

- (1) 操作分布特征
- 赋值语句占 38%-45%，核心是数据移动；
- IF 条件语句占 29%-43%，核心是序列控制；
- CALL 过程调用占 12%-15%，LOOP 循环占 3%-5%；
其中 CALL/RETURN、LOOP 是最耗时的操作，占机器指令权重的 32%-42%。
- (2) 操作数分布特征
- 标量变量占 55%，常量占 20%，数组/结构占 25%，绝大多数访问是局部标量变量，符合局部性原理。

- (3) 过程调用特征
- 过程调用非常耗时，依赖参数数量和嵌套深度；
- 大多数程序的调用嵌套深度有限，绝大多数变量是局部变量。
- 二八法则：程序 80% 的运行时间耗在 20% 的代码上。

最终结论：优化应聚焦于最常用、最耗时的特征：

- 1. 大量寄存器优化操作数访问；
 - 2. 精心设计流水线优化分支与调用；
 - 3. 简化 (精简) 指令集。
5. Large Register file and why to use it
大寄存器文件 (Large Register file) 是 RISC 的核心硬件设计，使用原因：

- (1) 赋值操作大多是简单数据移动，绝大多数访问是局部标量，目标是最小化寄存器-内存访问，减少内存访问延迟；
- (2) 寄存器是存储器层次结构的最顶层，访问速度远高于内存，大量寄存器可让更多变量常驻高速存储；
- (3) 过程调用的参数传递、局部变量存储均可在寄存器中完成，大幅降低过程调用的开销。
- 寄存器优化的两类方案：**
- 软件方案：依赖编译器分配寄存器，基于给定时间内最常用的变量分配，需要复杂的程序分析；
 - 硬件方案：增加物理寄存器数量，让更多变量可以驻留在寄存器中，是 RISC 的主流方案。

6. Register optimization (寄存器优化的指导思想) Considerations and solutions

寄存器优化的**核心指导思想**：最大化寄存器利用率，最小化内存访问。

核心考量 (Considerations)：

- 大量寄存器可减少内存访问；
- 全局变量：将最频繁使用的全局变量存放在寄存器；
- 局部变量：将最近使用的局部标量变量存放在寄存器；
- 过程调用会改变访问局部性，需要支持快速切换；
- 需要支持零开销的参数传递与返回。

解决方案 (Solutions)：

- 硬件方案：寄存器窗 (Register Windows)，通过硬件实现多组寄存器的快速切换与重叠，支持零开销参数传递；
- 软件方案：编译器寄存器分配 (如图着色算法)，在有限寄存器数量下最大化利用率。

7. Register windows (寄存器窗，重点) Organization

寄存器窗 (Register Windows) 是 RISC 大寄存器文件的核心组织方式，架构如下：

每个过程使用一组独立的小寄存器集合，每个寄存器集合包含三个区域：

- 参数寄存器** (Parameter Registers)：存放传入的参数，供当前过程读取；
 - 本地寄存器** (Local Registers)：存放当前过程的局部变量，仅当前过程可访问；
 - 暂存寄存器** (Temporary Registers)：用于传递参数给被调用过程，是调用者的输出区域。
- 核心优势：
- 过程调用时无需保存/恢复寄存器，仅切换窗口指针，开销极低；
 - 支持参数的零开销传递，无需数据移动；
 - 适配过程调用的局部性特征，局部变量常驻寄存器。
- 函数返回时切换回原寄存器组，自动恢复现场

8. Global variables (全局变量和局部变量如果分配寄存器)

全局变量与局部变量的寄存器分配规则：

- 局部变量**

- 分配到当前过程的寄存器窗的局部寄存器区；
- 过程调用时自动切换窗口，局部变量随窗口切换，无需保存/恢复；
- 仅当窗口满时，最旧窗口的局部变量才会保存至内存。

(2) **全局变量**

- 分配到专门的全局寄存器集合，所有过程共享；
- 频繁访问的全局变量常驻全局寄存器，不频繁访问的全局变量存放在内存；
- 由编译器负责全局变量的寄存器分配与调度。

两种选择：第一种是变量在 HLL 中被声明为全局的，并由编译器分配内存位置，所有引用这些变量的机器指令将使用内存引用操作数。这很简单。但是，对于频繁访问的全局变量，这种方式效率很低。

第二种是在处理器中合并一组全局寄存器。这些寄存器数量固定，可以用于所有的过程。需要增加硬件负担

核心原则：局部变量随过程窗口动态分配，全局变量固定共享分配，最大化高频变量的寄存器驻留率。

9. Overlapping Register Windows (寄存器窗的重叠结构，重点)

重叠寄存器窗 (Overlapping Register Windows) 是寄存器窗的核心设计，实现零开销参数传递：调用者的临时寄存器 (Temporary Registers) 与被调用者的参数寄存器 (Parameter Registers) 物理重叠，即二者是同一组物理寄存器。(不保存暂存：避免重复保存数据，提高上下文切换效率)

工作流程：

- 调用者将参数写入自己的临时寄存器区；
- 调用发生时，窗口指针切换到被调用者的窗口；
- 被调用者直接从自己的参数寄存器区读取参数，无需任何数据移动；
- 返回通过反向重叠传递，同样无需数据移动。

核心优势：完全消除过程调用的参数复制开销，大幅降低 CALL/RETURN 的执行时间，是 RISC 过程调用效率远高于 CISC 的核心原因。

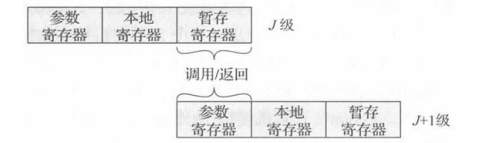


图 15-1 重叠寄存器窗口

10. Circular Buffer diagram (寄存器窗的环形结构，重点) CWP, SWP; CWP and SWP update in calling and return

寄存器窗以环形缓冲 (Circular Buffer) 结构组织，核心是两个指针：

- 核心指针定义
 - CWP (Current Window Pointer, 当前窗口指针)**：指向当前活动的寄存器窗口，标记当前正在执行的过程对应的窗口；
 - SWP (Saved Window Pointer, 已保存窗口指针)**：指向第一个需要保存到内存的窗口，标记环形缓冲中最旧的未保存窗口。
 - 调用 (Call) 时的指针更新
 - CWP++ (切换到下一个空闲窗口)；
 - 若 CWP == SWP (模 W，W 为窗口总数)：说明所有窗口已用完，保存最旧的窗口 (SWP 指向的窗口) 到内存，然后 SWP++，再执行调用；
 - 否则：直接切换窗口，执行调用，无需任何保存操作。
 - 返回 (Return) 时的指针更新
 - CWP-- (切换回调用者的窗口)；
 - 若 CWP == SWP (模 W)：说明有已保存的窗口需要恢复，从内存恢复最近保存的窗口，然后 SWP--，再执行返回；
 - 否则：直接切换窗口，执行返回，无需任何恢复操作。
- 核心规则**：N 个窗口的寄存器文件最多可容纳 N-1 个过程激活，窗口满时触发中断保存最旧的窗口。

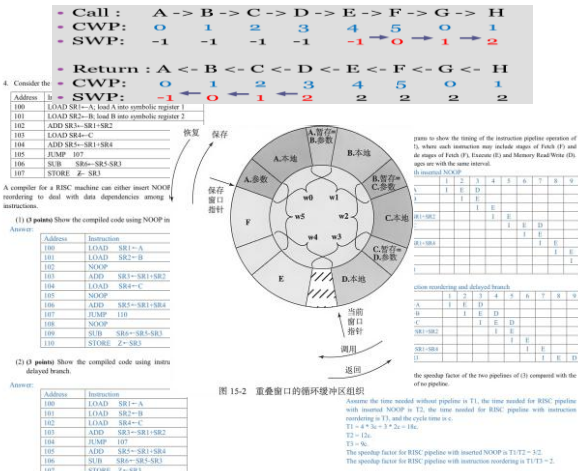


图 15-2 重叠窗口的循环缓冲区组织

11. Graph Colouring (变量在寄存器内的分配优化)

图着色 (Graph Colouring) 是编译器寄存器分配的核心算法，用于在有限物理寄存器下最大化寄存器利用率，原理如下：

- 问题转化
 - 将寄存器分配转化为图着色问题：
 - 节点：符号寄存器 (虚拟寄存器，对应程序中的变量)；
 - 边：两个符号寄存器在同一程序片段中同时活跃 (生命周期重叠) 则连边，表示二者不能共享同一个物理寄存器；
 - 颜色：物理寄存器，n 种颜色对应 n 个物理寄存器。
- 算法目标：用 n 种颜色给图着色，相邻节点颜色不同；无法着色的节点溢出到内存。
- 核心优势
 - 是最有效的编译器寄存器分配算法，可最大化寄存器利用率；
 - 适用于寄存器数量有限的场景，是软件寄存器优化核心技术；
 - 生命周期不重叠的变量可共享同一个物理寄存器，大幅减少寄存器需求。

图 15-4 是这个过程的一个简单示例。假设有一个 6 个符号寄存器的程序被编译成 3 个实际寄存器。图 15-4a 给出了每个符号寄存器有效使用的时间序列。水平虚线表示连续的指令执行。图 15-4b 给出了寄存器干扰图 (用阴影和条纹代替颜色)，并显示了一种可能的使用 3 种颜色的着色方案。由于符号寄存器 A 和 D 无干扰，编译可以把它们分配给物理寄存器 R1。类似地，符号寄存器 C 和 E 可以分配给寄存器 R3，还剩一个符号寄存器 F 没有着色，必须使用加载和存储来处理。

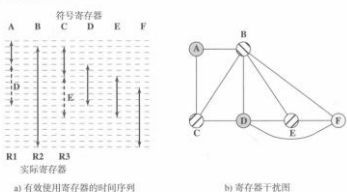


图 15-4 图着色方法

12. RISC Pipelining (RISC 流水线 重点)

RISC 指令集的设计天然适配流水线**标准 RISC 流水线**分为两类：

- 寄存器-寄存器运算指令：2 级流水线**
 - I 阶段 (Instruction fetch)：取指令；
 - E 阶段 (Execute)：执行 ALU 运算，寄存器输入输出；

所有运算指令均为 2 级，单周期完成，无内存访问。

为何不能将指令周期缩短一半：阶段时间不同，流水线开销，相关性冲突，内存访问冲突

- Load/Store 指令：3 级流水线**
 - I 阶段 (Instruction fetch)：取指令；
 - E 阶段 (Execute)：计算内存地址；
 - D 阶段 (Memory) 内存访问完成寄存器与内存之间的数据传输；

核心优势：指令格式统一，长度固定，译码简单，流水线控制复杂度低；

- 绝大多数指令为寄存器-寄存器操作无内存访问流水线停顿少；
- 单周期执行指令，流水线深度浅，分支惩罚小。

Effects of Pipelining

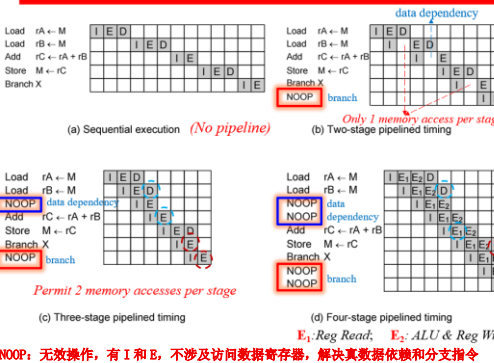


图 15-5 流水线的效果

(d) **重排序减少 NOOP 数量**：Load rA-M, Load rB-M, NOOP, BranchX, Add rC=rA+rB, Store M=rC **例题见第九页**

13. Optimized pipeline (流水线优化) How to use Delayed Branch; How to use Delayed Load; How to use NOOP

RISC 流水线的三类核心优化技术：**延迟分支延迟加载循环展开**

- 延迟分支** (Delayed Branch) **原理**：分支指令 (如 **JMP, BEQ**) 的跳转效果不在分支指令执行后立即生效，而是延迟到延迟槽 (分支后一条指令 (**找不到有用指令就用 NOOP**)) 执行完成后生效；延迟槽的指令无论分支是否跳转都会执行。

使用方法：编译器重新排列指令顺序，**将与分支无关的有用指令填充到延迟槽**，避免插入 NOOP 浪费周期；适用于无条件分支、调用、返回，条件分支需注意延迟槽指令不能修改分支条件。

普通分支指令：执行到它时，立即改变 PC，跳转到目标地址

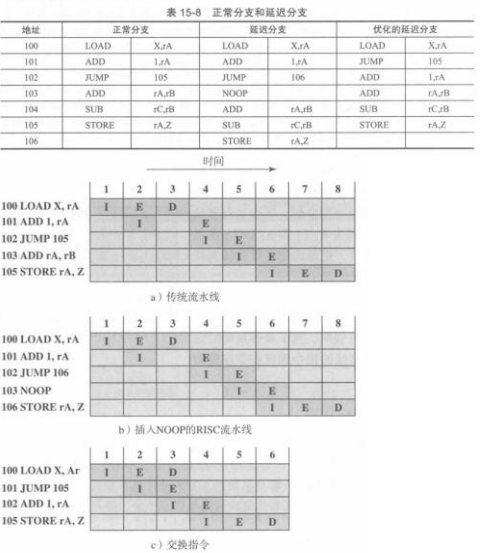


图 15-7 使用延迟分支

(2) **延迟加载** (Delayed Load)

原理：当执行 Load 时，CPU 立即将 R1 锁存状态，表示该 Reg 内容没有准备好 (正从内存读取中)，CPU 不会停顿，而是继续往下执行指令，直到遇到一条需要使用该寄存器时 CPU 会进入等待状态。

使用方法：编译器重新排列指令顺序，将不依赖加载结果的指令放在 Load 之后，隐藏加载延迟，消除流水线停顿。

(3) **NOOP 指令的使用**

当无法找到有用指令填充延迟槽/加载延迟槽时，插入 NOOP (空操作) 指令，保证流水线时序正确，避免数据冒险与控制冒险；NOOP 指令不执行任何操作，仅占用一个时钟周期，是流水线优化的兜底方案。

补充：SPARC: K 个窗口

全局寄存器：8 个 (0-7)；局部寄存器：每个窗口 8 个，共 8K 个；重叠寄存器：每个窗口 8 个 ins, 8 个窗口，由于寄存器重叠共 8K 个；每个窗口寻址 24 个寄存器，**总共物理寄存器数量：N=16K+8 个，K 个窗口构成的环形缓冲区最多支持 K-1 层嵌套**

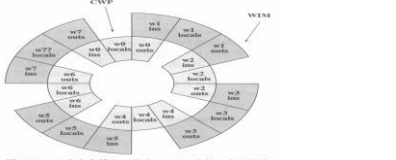


图 15-13 寄存器窗口形成 SPARC 中的一个寄存器文件

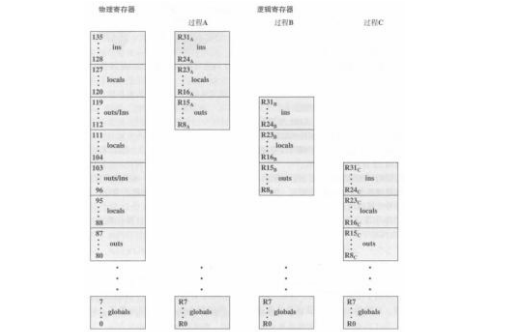


图 15-12 三个过程的 SPARC 寄存器窗口布局

Ch16 Superscalar Processors 知识点总结

1. Superscalar & Super-pipeline 定义

(1) Superscalar (超标量) 定义

超标量处理器是指在一个时钟周期内可以同时取指、译码、发射多条独立指令，通过复制多套功能单元，实现指令级并行的处理器架构。**核心特征**：同一时钟周期内启动多条指令执行，通过硬件并行提升吞吐量。

本质：在不同流水线中独立并发地执行指令

(2) Super-pipeline (超流水线) 定义

超流水线是指将流水线的每个阶段进一步拆分为多个更细的子阶段，提升内部时钟频率，在一个外部时钟周期内完成多个流水线操作的架构。核心特征：流水线深度更深，内部时钟频率更高，通过时间重叠提升吞吐量。

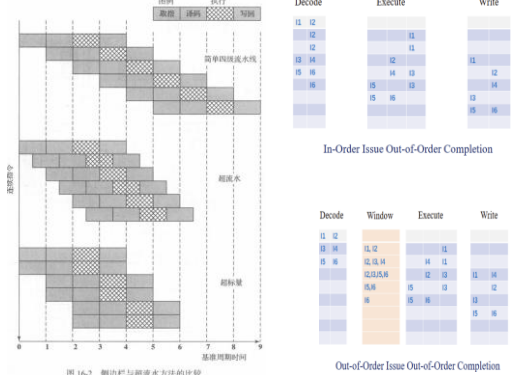


图 16-2 侧边栏与超流水线方法的比较

二者核心差异：

- 超标量：时间并行复制多套功能单元同一周期执行多条指令；
- 超流水线：时间并行，拆流水线阶段，提升时钟频率，同一周期完成多个子阶段操作。

2. Independent instructions 定义

独立指令 (Independent instructions) 是指指令之间不存在任何数据依赖、控制依赖或资源冲突，可以在不改变程序执行结果的前提下，同时发射、并行执行的指令。

核心判定标准：

1. 不存在真数据依赖 (RAW)：后一条指令不需要前一条指令的运算结果作为输入；
 2. 不存在控制依赖：指令之间不存在分支、跳转等控制流依赖；
 3. 不存在资源冲突：指令不需要同时访问同一硬件资源；
 4. 不存在输出依赖 (WAW) 和反依赖 (WAR)。
- 独立指令是超标量处理器实现指令级并行的基础，处理器通过识别独立指令实现并行发射与执行。

3. Instruction-level parallelism & Machine Parallelism 定义

(1) Instruction-level parallelism (ILP, **指令级并行**) 定义：指令级并行是指程序中指令可以被并行执行的程度，是程序本身的固有属性，由程序中**数据依赖 (RAW 真数据依赖，写后读)、控制依赖的频率以及操作延迟 (指令结果可以被用作后续操作数所需的时间) 决定**。核心是程序中独立指令的占比，反映了程序本身可挖掘的并行潜力。

(2) Machine Parallelism (**机器并行度**) 定义 见 10

机器并行度是指**处理器硬件能够利用指令级并行的能力**，是处理器的硬件属性，由两个因素决定：

1. 可以同时取指、执行的并行流水线数量；
2. 识别独立指令的机制的速度与复杂度。

本质：1. 寻找独立指令机制的速度和复杂度，需要找得快和准
2. 可并行取指令和执行指令的流水线数量
二者关系：

- 指令级并行是程序的潜力，机器并行度是处理器挖掘潜力的能力；
- 程序的指令级并行不足时，即使机器并行度再高也无法提升性能；
- 机器并行度不足时，即使程序有大量并行指令也无法充分利用。

4. Unnecessary dependency 可以被克服的因素

非必要依赖 (Unnecessary dependency) 是指并非程序逻辑必需，可以通过硬件或编译器技术消除的依赖，是超标量处理器可以克服的因素，主要包括两类：

(1) **输出依赖 (WAW, 写后写依赖)**

两条指令先后写入同一个寄存器，后写入的结果会覆盖先写入的结果，这种依赖并非程序逻辑必需，可以通过寄存器重命名消除。**A is after, 先后后面前面**

(2) **反依赖 (WAR, 写后读依赖)**

后一条指令写入前一条指令读取的寄存器，这种依赖是因为寄存器复用导致的，并非程序逻辑必需，可以通过寄存器重命名消除。

核心特点：

- 真数据依赖 (RAW) 和控制依赖是程序逻辑必需的，无法消除；
- 输出依赖和反依赖是寄存器复用导致的伪依赖，可以通过寄存器重命名、乱序执行等技术完全克服，是超标量处理器提升并行度的核心优化方向。

5. Superscalar limitation 重点

超标量处理器的核心限制 (五类依赖与冲突)：

- (1) **True data dependency (真数据依赖, RAW, 写后读, 流相关)**
后一条指令的输入是前一条指令的输出，是程序逻辑必需的依赖，无法消除；只能通过数据转发减少延迟，无法完全并行执行。

真数据相关 考虑如下序列^①：

ADD R4R, R2R ; 把 R2R 内容加上 R4R 内容的结果加到 R4R 寄存器
MOV R2R, EAX ; 把 R4R 内容复制到 R2R

第二条指令可以进行取指和译码，但不能执行，需等待第一条指令的执行。理由是第二条指令需要用到第一条指令产生的数据。这种情况被称为真数据相关 (也称为流相关或先写后读 [RAW] 相关)。

(2) **Procedural dependency (控制依赖, 过程依赖)**

两类因素：①分支指令：分支跳转改变指令流，分支后的指令无法与分支前的指令并行；②可变指令长度：指令需要译码后才能确定长度，无法同时取指多条指令；RISC 固定指令长度更适合超标量。

(3) **Resource conflicts (资源冲突)**

两条或多条指令同时访问同一硬件资源 (如 ALU、内存端口)，可以通过复制资源 (如多套 ALU、多端口内存) 解决。

(4) **Output dependency (输出依赖, WAW, 写后写)**

两条指令先后写入同一个寄存器，是伪依赖，可以通过寄存器重命名消除。

除了上述限制之外，还出现了一个新的相关性，之前我们将其称为输出相关 (也被称为写后写 [WAW] 相关)。下面的代码段展示了这种相关 (op 表示任意操作)：

```
11: R3 ← R3 op R5  
12: R4 ← R3 + 1  
13: R3 ← R5 + 1  
14: R7 ← R3 op R4
```

指令 12 无法在指令 11 之前执行，因为它需要 11 产生的寄存器 R3 中的结果；这是一个真数据相关的例子，如 16.1 节所述。类似地，14 必须等待 13，因为它要使用 13 产生的结果。那么，11 和 13 之间是什么关系？按照我们的定义，这里不存在数据相关。但是，如果 13 在 11 之前执行完成，那么 14 执行时就会拿到错误的 R3 的内容。因此，13 必须在 11 之后完成，以便产生正确的输出值。为了确保这一点，如果第三条指令的结果稍后会被需要更长时间完成的前序指令覆盖，那么它的发射必须停顿。

(5) **Antidepending (反依赖, WAR, 读后写)**

后一条指令写入前一条指令读取的寄存器，是伪依赖，可以通过寄存器重命名消除。

性。此外，还出现了一种新的相关性，我们之前将其称为反相关 (也称为读后写 [WAR] 相关)。前面要考虑的代码段展示了这种相关：

```
11: R3 ← R3 op R5  
12: R4 ← R3 + 1  
13: R3 ← R5 + 1  
14: R7 ← R3 op R4
```

在指令 12 取出其操作数并开始执行之前，指令 13 不能完成执行。其原因是 13 会更新寄存器 R3，而它又是 12 的源操作数。使用反相关一词，是因为该约束类似于真数据相关，但是相反的：与第二条指令使用的值由第一条指令产生相反，第二条指令会破坏第一条指令使用的值。

6. Hardware techniques in a superscalar processor : 超标量处理的硬件技术

超标量处理器的三类核心硬件技术：

(1) **Duplication of resources (资源复制)**

复制多套功能单元 (如多套整数 ALU、浮点运算单元、Load/Store 单元)，解决资源冲突问题，让多条同类指令可以同时执行。

(2) **Out-of-order issue (乱序发射)**

将译码阶段与执行阶段解耦，使用指令窗口缓存已译码的指令：当功能单元空闲时，从指令窗口中选择无依赖、无冲突的指令优先发射执行，不严格按照程序顺序发射，提升功能单元利用率。(指令窗口的作用)

(3) **Renaming (寄存器重命名)**

动态分配物理寄存器，将程序中的逻辑寄存器映射到不同的物理寄存器，消除输出依赖 (WAW) 和反依赖 (WAR)，解决伪依赖导致的流水线停顿，提升乱序执行的并行度。

7. Instruction Issue Policy (难点)

指令发射策略是处理器发射、执行、写回指令的协议 (哪条指令在哪个周期送往哪个执行单元地规则)，分为三类：

(1) **In-Order Issue In-Order Completion (顺序发射顺序完成)**

- 严格按照程序顺序发射指令，严格按照程序顺序写回结果；
- 效率最低指令遇到依赖或冲突时必须停顿后续指令全部等待；
- 实现最简单，无需复杂的依赖检测与重排序逻辑。

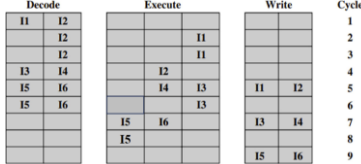
特点：虽然 1 个周期中取出多条指令，但是连续的

① 两条指令同时需要硬件 (除法器，ALU, Reg) 引起冲突，后面指令只能停顿

② 第 2 条指令需要第 1 条指令的结果，第 1 条指令耗时长，第 2 条指令须等待，独立的第 3 条指令须停顿

③ 一旦遇到分支无法跨过分支去取后面指令只能等待分支结果

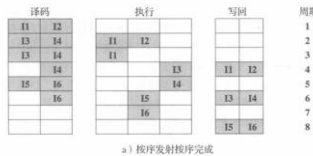
考虑如图 16-14 所示的“按序发射/按序完成”的执行序列。
a. 找出 12 直到第 4 个周期才能进入执行阶段的最有可能的原因。“按序发射/无序完成”或“无序发射/无序完成”能解决这个问题吗？如果可以，是哪一个能解决这个问题？
b. 找出 16 直到第 9 个周期才能进入写回阶段的最有可能的原因。“按序发射/无序完成”或“无序发射/无序完成”能解决这个问题吗？如果可以，是哪一个能解决这个问题？



见左图

- a. The execution of 12 requires the result of 11 execution. This is a true data dependency. Neither of the two policies can fix this.
- b. This is the result of the policy of “in-order issue and in-order completion”. Both of the two policies can fix this.

- 11 的执行需要两个周期。
- 13 和 14 对同一个功能单元有冲突。
- 15 依赖于 14 生成的值。
- 15 和 16 有功能单元冲突。



a) 按序发射按序完成

(2) **In-Order Issue Out-of-Order Completion (顺序发射乱序完成)**

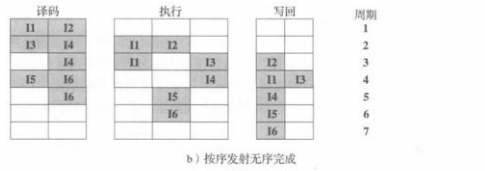
- 严格按照程序顺序发射指令，但执行完成的顺序可以与程序顺序不同；
- 多个指令可以同时处于执行阶段，执行快的指令先写回；
- 会产生输出依赖 (WAW)，需要依赖检测机制，效率高于顺序完成。

特点：任意时刻有任意数量指令被执行，最大是所有功能单元所支持的最大并行度；指令执行完后，功能单元释放，后面指令继续使用

(3) **Out-of-Order Issue Out-of-Order Completion (乱序发射乱序完成)**

- 译码与执行解耦，指令窗口缓存已译码指令，无依赖无冲突的指令可以优先发射，不按程序顺序；
- 执行完成顺序也与程序顺序不同；
- 效率最高，会产生输出依赖和反依赖，需要寄存器重命名、指令窗口、重排序缓存等复杂硬件支持，是现代超标量处理器的主流策略。

特点：有更多的指令发射，降低了流水线停顿的概率



b) 按序发射无序完成



c) 无序发射无序完成

图 16-4 (续)

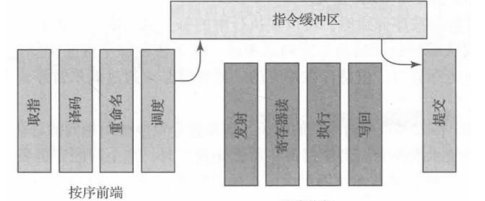


图 16-5 无序发射无序完成的结构

8. Registers renaming (重点)

寄存器重命名是超标量处理器消除伪依赖的核心技术：

(1) 核心原理

程序中的逻辑寄存器 (如 x86 的 EAX、EBX) 是程序员可见的，数量有限；处理器内部有大量物理寄存器，动态将逻辑寄存器

映射到不同的物理寄存器，同一逻辑寄存器在不同指令中映射到不同物理寄存器，消除寄存器复用导致的伪依赖。

- (2) 消除的依赖类型 **RAW 消除不了 左边写入 右边读出**
- 输出依赖 (WAW)：两条指令写入同一逻辑寄存器，映射到不同物理寄存器，不存在覆盖问题；
- 反依赖 (WAR)：后写的指令映射到新的物理寄存器，不影响前读指令的原始物理寄存器值。

(3) **示例 (注: I2 覆盖了 I1, 故 RAW 无 I1I5)**

- a. Identify the write-read, write-write, and read-write dependencies in the following instruction sequence:
- ```
I1: R1 = 100
I2: R1 = R2 + R4
I3: R2 = R4 - 25
I4: R4 = R1 + R3
I5: R1 = R1 + 30
```
- b. Rename the registers from part (a) to prevent dependency problems. Identify references to initial register values using the subscript "a" to the register reference.

WAW: I1I2, I1I5, I2I5  
RAW: I2I4, I2I5  
WAR: I2I3, I2I4, I3I4, I4I5

I1: R1a = 100  
I2: R1b = R2a + R4a  
I3: R2b = R4a - 25  
I4: R4b = R1b + R3a  
I5: R1c = R1b + 30

### 9. Dealing with branch in superscalar processors (使用预测技术, 不使用 delayed branch 技术)

**超标量处理器的分支处理策略:**

- (1) **为什么不使用延迟分支** (Delayed Branch)  
延迟分支是 RISC 单发射流水线的技术，通过在分支后填充一条无关指令来隐藏分支延迟；但超标量处理器需要在分支延迟槽填充多条无关指令，且容易产生指令依赖，延迟分支的收益极低，因此超标量处理器不使用延迟分支技术。
- (2) 回归到 **超标量处理器的分支处理技术: 分支预测**
  - 静态分支预测: 基于固定规则预测分支是否跳转，如向后分支预测跳转、向前分支预测不跳转；
  - 动态分支预测: 基于分支的历史执行记录预测分支方向，包括 1 位预测器、2 位饱和计数器预测器、分支历史表 (BHT)、分支目标缓冲 (BTB) 等；
  - 推测执行: 按预测方向预取、译码、执行指令，分支结果确定后，预测正确则提交结果，预测错误则回滚状态、清空流水线。

核心优势: 分支预测技术可以有效隐藏超标量处理器的分支延迟，是现代超标量处理器的标准分支处理方案。

### 10. Machine Parallelism evaluation (评估)

**机器并行度的性能评估结论** (基于仿真实验)：

- (1) 核心优化技术的收益
  - 资源复制 (增加功能单元)：可以提升并行度，但单独使用收益有限；
  - 乱序发射: 可以提升功能单元利用率，但单独使用受伪依赖限制；
  - 寄存器重命名: 可以消除伪依赖，大幅提升乱序执行的并行度，是收益最高的优化技术；
  - 三者结合 (资源复制+乱序发射+寄存器重命名)：性能提升最显著，是现代超标量处理器的标准配置。
- (2) 指令窗口大小的影响
  - 指令窗口 (Instruction Window) 是乱序发射中缓存已译码指令的缓冲区；
  - 窗口大小小于 8 时，无法充分挖掘指令级并行，性能提升有限；
  - 窗口大小达到 16、32 时，指令级并行的挖掘更充分，性能提升显著；
  - 窗口越大，硬件复杂度越高，需要平衡性能与成本。
- (3) 分支的影响

程序中的分支会显著降低机器并行度的收益，分支预测准确率越高，机器并行度的实际收益越接近理论值。

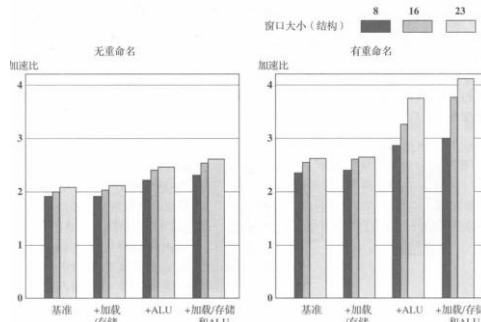


图 16-6 无程序相关的各种机器组成结构的加速比

### 11. Superscalar implementation (实现步骤)

**超标量处理器的完整实现步骤:** 超标量执行

- 同时取指多条指令: 基于分支预测，一个时钟周期内取指多条连续指令；
  - 指令译码与依赖检测: 译码所有取到的指令，检测指令之间的真数据依赖、控制依赖；
  - 寄存器重命名: 将逻辑寄存器映射到物理寄存器，消除输出依赖和反依赖；
  - 指令发射: 从指令窗口中选择无依赖、无资源冲突的指令，发射到对应的功能单元；
  - 并行执行: 多套功能单元同时执行发射的指令；
  - 结果缓存: 执行完成的结果暂存到重排序缓存 (ROB)，不直接写回架构寄存器；
  - 按序提交: 按照程序原始顺序，将重排序缓存中的结果依次写回架构寄存器和内存，保证程序执行结果正确。
- 核心原则: 乱序执行，按序提交，保证程序语义正确。

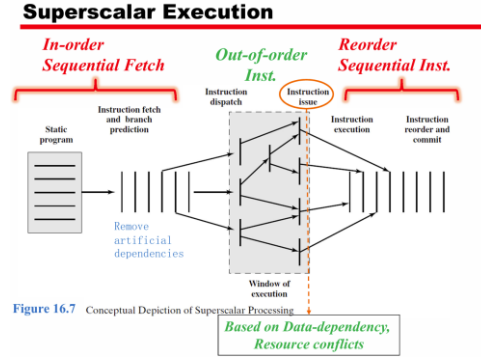


Figure 16.7 Conceptual Depiction of Superscalar Processing

### 12. Commit (定义)

**提交 (Commit)**，也称为 Retire，退休) 是超标量乱序处理器的最后一步，定义如下：  
将乱序执行完成的结果，按照程序的原始顺序，永久写入架构可见的寄存器和内存的过程。

核心作用:

- 保证程序语义正确: 即使指令乱序执行，最终的状态更新严格按照程序顺序，与顺序执行的结果完全一致；
- 支持推测执行: 分支预测错误时，未提交的推测执行结果可以直接丢弃，不会修改程序状态；预测正确时，再按序提交结果；
- 处理异常: 指令执行产生异常时，只有该指令到达提交阶段时才会触发异常，保证异常按照程序顺序处理。

核心流程:

- 执行完成的结果暂存在重排序缓存 (ROB) 中；
- 按照程序顺序检查 ROB 头部的指令是否执行完成、无异常、

分支预测正确；

- 满足条件则将结果写回架构寄存器/内存，完成提交，ROB 指针后移。

### Ch20 Control Unit Operations 知识点总结

**控制单元:** 是 CPU 的指挥中心，它不负责计算数据，而是负责解读指令，并指挥其他部件 (ALU, Regs, M) 协调工作，执行指令。向处理器和系统总线发送少量控制信号来控制一切

#### 1. Micro-Operations 定义

**微操作** (Micro-Operations  $\mu OP$ ) 是 CPU 执行指令时分解出的最基本、原子级的操作，是 CPU 的最小执行单元。  
**层级关系:** 程序执行  $\rightarrow$  指令周期 (取指/间址/执行/中断)  
 $\rightarrow$  每个周期分解为多个步骤  $\rightarrow$  每个步骤对应一个微操作。

**核心特点:**

- 原子性: 每个微操作完成一个极小的、不可再分的硬件动作；
- 时序性: 微操作按时钟周期顺序执行，遵循严格的先后顺序；
- 并行性: 无冲突的微操作可在同一时钟周期内并行执行；
- 硬件驱动: 每个微操作由控制单元发出的控制信号激活。

#### 2. Define in micro-operations 常见指令分解为微操作 (重点)

所有指令周期均可分解为 **微操作序列**，核心指令的微操作分解如下: (**红色字体为激活控制信号**)

##### (1) Fetch (取指周期)

t1: MAR  $\leftarrow$  (PC) C2  
t2: MBR  $\leftarrow$  (memory) C0, C5, CR  
PC  $\leftarrow$  (PC) + 1 (假设指令长度 I=1)  
t3: IR  $\leftarrow$  (MBR) C4

##### (2) Indirect (间址周期)

t1: MAR  $\leftarrow$  (IR address)/(IR 地址字段) C8  
t2: MBR  $\leftarrow$  (memory) C0 C5 CR  
t3: IR 地址字段  $\leftarrow$  (MBR 地址字段) C4

##### (3) Interrupt (中断周期)

t1: MBR  $\leftarrow$  (PC) C1  
t2: MAR  $\leftarrow$  保存地址  
PC  $\leftarrow$  中断服务程序入口地址  
t3: memory  $\leftarrow$  (MBR) C0 C12 CW

##### (4) 运算指令

ADD R1, X:  
t1: MAR  $\leftarrow$  (IR 地址) C8  
t2: MBR  $\leftarrow$  (memory) C0, C5, CR  
t3: R1  $\leftarrow$  R1 + (MBR) C6 C7 C9

AND/SUB/MUL: 结构与 ADD 一致，仅 ALU 操作不同  
求补 Complement AC:

t1: AC  $\leftarrow$   $\overline{AC}$  C7 C9

##### (5) 栈操作 (取址同上, 执行如下) rA 为寄存器, X 为地址

PUSH rA:  
t1: MBR  $\leftarrow$  (rA)  
SP  $\leftarrow$  SP-1;  
t2: MAR  $\leftarrow$  (SP);  
t3: memory  $\leftarrow$  (MBR)  
SP  $\leftarrow$  SP+1

##### (6) 跳转与调用

ISZ X (增量为零跳过): t1: MAR  $\leftarrow$  (IR 地址); t2: MBR  $\leftarrow$  (memory); t3: MBR  $\leftarrow$  MBR+1; t4: memory  $\leftarrow$  (MBR), 若 MBR=0 则 PC  $\leftarrow$  PC+1

BSA X (跳转并保存地址): t1: MAR  $\leftarrow$  (IR 地址), MBR  $\leftarrow$  (PC); t2: PC  $\leftarrow$  (IR 地址), memory  $\leftarrow$  (MBR); t3: PC  $\leftarrow$  PC+1

CALL X (栈方式): t1: MBR  $\leftarrow$  (PC), SP  $\leftarrow$  SP-1; t2: MAR  $\leftarrow$  (SP); t3: memory  $\leftarrow$  (MBR), PC  $\leftarrow$  X

Return (栈方式): t1: MAR  $\leftarrow$  (SP); t2: MBR  $\leftarrow$  (memory); t3: PC  $\leftarrow$  (MBR), SP  $\leftarrow$  SP+1

##### (7) 补充

Load X to AC:  
t1: MAR  $\leftarrow$  (IR address)/(IR 地址字段) C8  
t2: MBR  $\leftarrow$  (memory) C0 C5 CR

t3: AC  $\leftarrow$  (MBR) C10

Store AC to X:

t1: MAR  $\leftarrow$  (IR address)/(IR 地址字段) C8

t2: MBR  $\leftarrow$  (AC) C11

t3: memory  $\leftarrow$  (MBR) C0 C12 CW

JUMP:

t1: PC  $\leftarrow$  (IR address) C3

JUMP if AC=0:

t1: if (AC)=0 then PC  $\leftarrow$  (IR address) C3

#### 3. MAR/MBR/PC/IR 作用

**四个核心寄存器**的作用:

- (1) **MAR** (Memory Address Register 内存地址寄存器)
  - 连接系统地址总线，存放要访问的内存地址；
  - 所有内存读写操作的地址都通过 MAR 传递，是 CPU 与内存之间的地址接口。
- (2) **MBR** (Memory Buffer Register, 内存缓冲寄存器)
  - 连接系统数据总线，存放从内存读取的数据或要写入内存的数据；
  - 是 CPU 与内存之间的数据接口，所有内存数据交换都经过 MBR。
- (3) **PC** (Program Counter, 程序计数器)
  - 始终存储下一条要执行的指令的地址；
  - 每取完一条指令后自动自增，控制指令执行的 **顺序**，是程序流的核心控制器。
- (4) **IR** (Instruction Register, 指令寄存器)
  - 存放当前正在执行的指令；
  - 控制单元通过解析 IR 中的操作码和地址字段，生成对应的控制信号，驱动微操作执行。

#### 4. Instruction cycle code (ICC) (仅在硬件控制单元使用)

**指令周期码 (ICC)** 是硬布线控制单元中用于标记当前指令周期阶段的 2 位寄存器，仅在硬布线控制单元中使用，用于控制周期切换。

**编码定义:**

- 00: Fetch (取指周期) 01: Indirect (间址周期)
- 10: Execute (执行周期) 11: Interrupt (中断周期)

**工作流程:**

- 初始 ICC=00，执行取指周期；
- 取指完成后，判断是否为间接寻址，是则 ICC=01 执行间址周期，否则 ICC=10 执行执行周期；
- 执行周期完成后，判断是否有中断，是则 ICC=11 执行中断周期，否则 ICC=00 回到取指周期。

核心作用: 通过简单的状态编码，驱动硬布线逻辑生成对应周期的控制信号，是硬布线控制单元的核心状态标记。  
**e.g.** 一个简单的处理器在其指令周期中具有四个主要阶段: 取指 (fetch)、间址 (indirect)、执行 (execute) 和中断 (interrupt)。在硬连线实现中，使用两个 1 位 (1-bit) 标志来指示当前的阶段。

**a. 为什么需要这些标志? b. 为什么在微程序控制单元中不需要它们?**  
**解答:** a. 这些标志代表输入到控制单元逻辑的布尔变量。它们与时间输入及其他标志一起，共同决定控制单元的输出。  
b. 指令周期的阶段隐含在微程序的组织结构中。微程序存储器中的特定位置对应于这四个阶段中的每一个。

#### 5. Functions of Control Unit 控制单元的两个功能

功能需求: 控制单元必须执行的那些功能  
基本功能要素: ALU, 寄存器, 内部数据通路, 外部数据通路, 控制单元

**控制单元的核心功能**分为两类:

- (1) **Sequencing (时序控制)**
  - 驱动 CPU 按正确的顺序执行一系列微操作；
  - 控制指令周期的切换、微操作的先后顺序，保证程序按逻辑顺序执行; **执行方式: 使用指令序列**
  - 由时钟、ICC、指令操作码共同驱动。





log2(n) 位;

- 并行性：仅支持单条或少量微操作执行，并行度低；
- 译码：需要额外的译码电路将功能码转换为控制信号；
- 优点：微指令长度短，控制存储器占用空间小；
- 缺点：执行速度慢，并行度低。

6. 微程序控制单元结构及工作过程（重点）

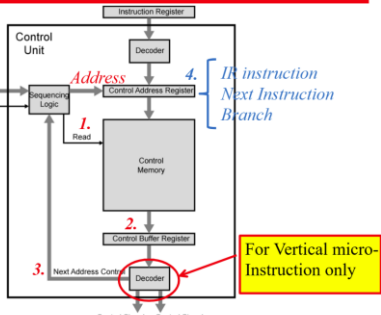
微程序控制单元的四个核心组件：

- (1) **Control Memory (控制存储器, CM)**
  - 是高速只读存储器 (ROM)，存储所有微程序；
  - 每个存储单元存放一条微指令，按地址寻址；
- 是**微程序的存储载体**，速度远高于主存，接近寄存器速度。
- (2) **Control Address Register (控制地址寄存器, CAR)**
  - 存放**下一条要读取的微指令的地址**；
  - 功能类似机器指令的 PC，控制微程序的执行顺序。
- (3) **Control Buffer Register (控制缓冲寄存器, CBR)**
  - 存放**当前正在执行的微指令**；
  - 功能类似机器指令 IR，解析微指令的控制信号域和下地址域。
- (4) **Sequencing Logic (时序逻辑)**
  - 驱动微指令的取指、执行流程；
  - 根据标志位、IR 操作码、CBR 的跳转条件，生成下一条微指令的地址，加载到 CAR。

完整工作过程：

- 1) 为了执行一条指令，排序逻辑单元向控制存储器发出 READ 命令，CAR 中的地址送入控制存储器；
- 2) 控制地址寄存器指定地址中的字被读进控制缓冲寄存器。
- 3) 控制缓冲寄存器的内容生成控制信号以及排序逻辑单元的一个地址信息。
- 4) 排序逻辑单元根据来自控制缓冲寄存器的伪指令地址、跳转条件、标志位、IR 操作码，生成下一条微指令地址，更新 CAR；
- 5) 重复上述步骤，直到微程序执行完成。

Functioning of Microprogrammed Control Unit



14.11 考虑一个流程指令流水线的，长度为  $n$  的指令序列。令  $p$  为遇到条件或无条件分支指令的概率， $q$  为执行分支指令  $i$  会导致跳转到非连续地址的概率。假设当指令  $i$  从最后一个阶段出来时，每一次这样的跳转都需要清除流水线，即破坏所有正在处理中的指令。考虑这些概率，修改式 (14.1) 和式 (14.2)。

遇到指令不跳转  $1-pq$ , 跳转概率  $pq$

Handwritten equations for pipeline analysis:

$$T_{n,n} = kC + (p+q)C(n-1)C + (1-p-q)C - pq(n-1)C$$
$$S_k = \frac{T_{n,n} - T_{n-1,n-1}}{n - (n-1)} = \frac{kC + (p+q)C(n-1)C + (1-p-q)C - pq(n-1)C - (kC + (p+q)C(n-2)C + (1-p-q)C - pq(n-2)C)}{1} = kC + (p+q)C(n-1)C + (1-p-q)C - pq(n-1)C$$
$$S_k = \frac{T_{n,n} - T_{n-1,n-1}}{n - (n-1)} = \frac{kC + (p+q)C(n-1)C + (1-p-q)C - pq(n-1)C - (kC + (p+q)C(n-2)C + (1-p-q)C - pq(n-2)C)}{1} = kC + (p+q)C(n-1)C + (1-p-q)C - pq(n-1)C$$

7. Next Address Decision 下一个微指令地址的三种形式下一条微指令地址的三种生成方式：

1. **顺序地址**：CAR 自增 1，执行微程序的下一条微指令，是最常见的方式，对应微程序的顺序执行；
2. **微指令指定的分支/跳转地址**：根据 CBR 中的下地址域，直接加载跳转目标地址到 CAR，对应微程序的无条件分支；
3. **映射指令起始地址（指令译码的启示地址）**：根据 IR 中的机器指令操作码，映射生成对应微程序的起始地址，加载到 CAR，对应机器指令译码后进入对应微程序的入口。
  - 获取下一条指令 控制地址寄存器加 1。

534 第六部分 控制单元

- 跳转回基于跳转微指令的新例程 把控制缓冲寄存器的地址字段加载到控制地址寄存器。
  - 跳转回机器指令例程 根据 IR 中的操作码加载控制地址寄存器。
- 三种方式由时序逻辑根据当前状态选择：取指微程序结束后用映射地址进入执行微程序，顺序执行时用自增地址，遇到分支时用跳转地址。

Wilkes 提出的结构如图 21-5 所示。该系统的核心是部分填充二极管的矩阵。在一个机器周期中，矩阵中的一行被脉冲激活。这将在二极管所在的地方产生信号（图中用一个点表示）。该行的第一部分生成的控制信号控制处理器的操作。第二部分生成在下一个机器周期中将要脉冲激活的地址行。因此，矩阵的每一行就是一条微指令，矩阵的布局就是控制存储器。

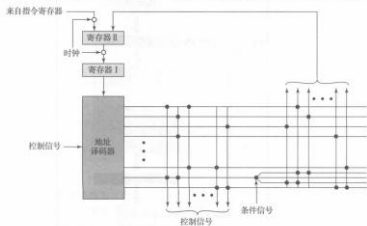


图 21-5 Wilkes 微程序控制单元

8. Control unit functions 控制单元功能

微程序控制单元的核心功能与硬布线控制单元完全相同，仅实现方式不同：

- (1) **Sequencing (时序控制)**
    - 保证微指令按正确的顺序执行；
    - 由 CAR、时序逻辑实现，控制微程序的执行流程、分支跳转、微程序入口映射。
  - (2) **Execution (执行控制)**
    - 生成每个微操作对应的控制信号；
    - 由 CBR 的控制信号域译码电路实现，驱动微操作执行。
- 核心差异：硬布线控制的时序和执行由组合逻辑电路实现，微程序控制的时序和执行由存储的微程序实现。

9. Sequencing Techniques 排序技术的三种地址

- 微指令的三种地址定义格式：**A1 下一条微指令地址**
- (1) **Two address fields (双地址域) A2 跳转目标地址**
    - 微指令包含两个地址域：跳转地址 1、跳转地址 2；
    - 由分支逻辑根据标志位选择其中一个地址加载到 CAR；
    - 优点：分支逻辑简单；缺点：微指令长度长，占用空间大。
  - (2) **Single address field (单地址域)**
    - 微指令仅包含一个跳转地址域；
    - 分支逻辑二选一：条件满足则加载跳转地址，不满足则 CAR 自增 1；**地址字段，指令寄存器编码，下一个顺序地址**
    - 优点：微指令长度短；是主流的地址格式。
  - (3) **Variable format (可变地址格式) 水平垂直间的平衡**
    - 微指令分为两种格式：非分支型微指令无地址域，CAR 自增；分支型微指令包含地址域和分支控制域，一个指示位 (indicating bit)，一个或多个控制字段，以及一个或多个地址字段 / 地址数量是可变的：用一位来指示该微指令是否有地址字段。**地址字段的长度或具体含义不是固定不变的，根据控制位既可以被用作下一个微指令的地址，也可其他用途**

- 优点：非分支微指令长度更短，节省控制存储空间，节省寻址位数；
- 缺点：需要额外的格式判断逻辑，控制复杂度略高，每个分支微指令都会占用一个完整的周期。

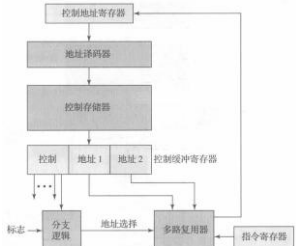


图 21-6 分支控制逻辑：双地址字段

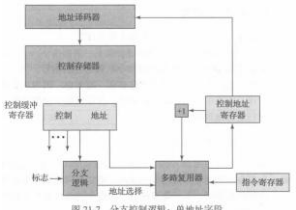


图 21-7 分支控制逻辑：单地址字段

另一种方法是提供两种完全不同的微指令格式 (图 21-8)。一位表示正在使用的格式。在一种格式中，其余的位被用于激活控制信号。在另一种格式中，一些位驱动分支逻辑模块，其余位提供地址。对于第一种格式，下一个地址要么是下一个顺序地址，要么是从指令寄存器派生的地址。对于第二种格式，可以指定条件分支或无条件分支。这种方法的一个缺点是每个分支微指令都会占用一个完整的周期。对于其他方法，地址生成与控制信号生成在同一个周期中，从而最大限度地减少了控制存储器的访问。

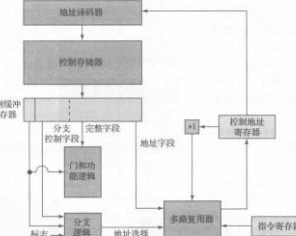


图 21-8 分支控制逻辑：可变格式

10. Encoding for execution 执行编码方式

控制信号域的四类编码方式：

- (1) **Direct/indirect encoding (直接/间接编码)**
  - 直接编码：每个控制域直接译码生成控制信号，一级译码，速度快；
  - 间接编码：一个控制域的译码结果控制另一个控制域的译码，二级译码，压缩率高。
- (2) **Packed/unpacked (压缩/非压缩)**
  - Packed (压缩)：控制信号编码压缩，相同位数包含更多控制信息；
  - Unpacked (非压缩)：控制信号不编码，每位对应一个控制信号，直接输出。
- (3) **Functional encoding/Resource Encoding (功能/资源编码)**
  - 功能编码：一个域对应一个功能，每个编码对应一个资源，如 ALU 操作域；
  - 资源编码：一个域对应一个资源，每个编码对应资源的一个操作。

21.3.1 微指令分类法

微指令可按各种方法来分类

- 垂直/水平
- 压缩/非压缩
- 硬/软微编程
- 直接/间接编码



11. IR 到 CAR 的映射方法

机器指令到微程序的映射过程：

1. 取指周期完成后，机器指令加载到 IR；
  2. 时序逻辑提取 IR 中的操作码字段；
  3. 通过映射电路（通常是 ROM 或组合逻辑），将 IR 中操作码转换为对应微程序在控制存储器中的起始地址；（第一个译码器指令译码器功能）第二个译码器（微指令译码器）功能：将从 CM 读出的、暂存在 CBR 的微指令解码为实际的硬件控制信号
  4. 将起始地址加载到 CAR，开始执行对应机器指令的微程序。
- 核心作用：实现机器指令到微程序的跳转，是连接机器指令层和微指令层的桥梁，每个操作码唯一对应一个微程序入口地址。
12. 子域指定动作的方法
- 将微指令的控制域划分为多个独立子域，每个子域控制一组互斥的动作：

1. 子域划分原则：同一子域内的动作是互斥的，不能同时执行；不同子域的动作是独立的，可以同时执行；**上取整**
  2. 每个子域有自己的编码，**n 个互斥动作编码为 log2(n) 位**；
  3. 每个子域独立译码，生成对应的控制信号组；
- 控制域划分为 ALU 操作子域、寄存器传输子域、内存操作子域：
- ALU 子域编码：00=加法、01=减法、10=与、11=或；
  - 寄存器子域编码：00=PC→MAR、01=MBR→IR、10=R1→MBR；
  - 内存子域编码：00=读、01=写、10=无操作；
- 三个子域的动作可以并行执行，大幅提升编码效率。

13. 微指令各域的解析方法

微指令各域的解析流程：

1. 控制信号域解析：
  - 按子域拆分控制信号域；
2. 跳转条件域解析：
  - 提取跳转条件编码；
  - 将条件编码与当前 CPU 标志位（零标志、进位、溢出）对比；
  - 输出分支选择信号，决定是否跳转。
3. 下地址域解析：
  - 若分支选择信号为跳转，提取下地址域的地址，加载到 CAR；
  - 若分支选择信号为不跳转，则 CAR 自增 1，执行下一条顺序微指令。

**e, g.** 考虑单地址字段控制单元。假设控制存储器的宽度为 24 位。微指令格式的控制部分被划分为两个字段。一个 13 位的微操作字指定将要执行的微操作。一个地址选择字段根据标志指定引起微指令分支的条件。总共有 8 个标志。

**a.** 地址选择字段有多少位？**b.** 地址字段有多少位？**c.** 控制存储器的大小是多少？

**解答：微操作字段 (Micro-operation field)** 题目已知为 13 位。**地址选择字段 (Address selection field / Condition field)**：用于选择根据哪个标志 (Flag) 来进行分支判断。**地址字段 (Address field)**：指向下一条微指令在控制存储器中的地址。微指令总长度 = 微操作字段 + 地址选择字段 + 地址字段 = 24 位。控制存储器 (Control Memory) 的大小取决于两个因素：1. 它能容纳多少个字 (Words)：这由地址字段的长度决定。地址字段有 8 位，意味着它可以寻址 2^8 个不同的内存位置。2. 每个字有多长：题目已知控制存储器的宽度（即每个字的长度）是 24 位。**a.** 指定 8 个标志中的 1 个需要 3 位。**b.** 24 - 13 - 3 = 8 **c.** 2^8 = 256 个字 × 24 位/字 = 6144 位。

**e, g.** 我们希望为每个机器指令例程 (machine instruction routine) 提供 8 个控制字 (control words)。机器指令操作码 (opcode=Routine) 有 5 位，控制存储器有 1024 个字。请建议一种从指令寄存器到控制地址寄存器的映射方案。

**解答：**因为 2^10 = 1024，所以控制存储器的地址长度必须是 10 位。为了保证每个例程有 8 个字的空間，我们映射出的 10 位地址的最低 3 位必须是 000。操作码是区分不同指令的唯一标识，所以这 5 位必须包含在控制地址中。最高位补两个 0，中间是 5 位操作码 XXXX，最低位是三个 0 **简单映射方案 00XXXX0000**

**e, g.** 将使用一种编码微指令格式 (encoded microinstruction format)。请说明如何将一个 9 位的微操作字段划分为多个子域，以指定 46 种不同的动作 (different actions)。(有效动作，减去全 0，NOP 无操作)  
**解答：**一个 5 位的字段可以产生 2^5 - 1 = 31 种不同的控制信号组合。一个 4 位的字段可以产生 2^4 - 1 = 15 种不同的组合，总共为 46 种。



### 21.3.2 微指令编码

实际上，微编程控制单元的设计没有使用纯无编码或水平微指令格式。至少要使用某种程度的编码来减少控制存储器宽度，并简化微程序设计的任务。

编码的基本技术如图 21-11a 所示。微指令被组织成一组字段。每个字段包含一个编码，其译码后会激活一个或多个控制信号。

让我们考虑一下这种布局的含义。当执行微指令时，每个字段被译码并生成控制信号。所以， $N$  个字段就指定了  $N$  个同步操作。每个动作都会激活一个或多个控制信号。通常，但并不是，我们想要设计这样一种格式，即每个控制信号被不超过一个的字段激活。但是，显然，每个控制信号都必须至少被一个字段激活。

现在考虑单个字段。一个字段由  $L$  位构成。它可以包含  $2^L$  个编码中的一个，其中的每一个都可以编码为不同的控制信号模式。因为，一次只能有一个编码出现在字段中，所以这些编码是互斥的，由此，它们所导致的操作也是互斥的。

现在可以用简单的术语来说明微指令格式的设计：

- 将格式组织成单独的字段。也就是说，每个字段描述一组动作（控制信号的模式）。那么就可以为此指定一个字，其中的每个编码指定一个不同的源。资源编码将把机器看作由一组独立资源组成，每个资源（如 I/O、内存、ALU）占用一个字。
- 定义每个字段，使得该字段指定的微操作是互斥的。也就是说，一次只能为给定字段指定一个操作。

有两种方法可以把编码后的微指令组织成字段：功能型和资源型。功能型编码方法标识机器内的功能，并根据功能类型指定字段。例如，如果可以使用各种源把数据传递给累加器，那么就可以为此指定一个字，其中的每个编码指定一个不同的源。资源型编码把机器看作由一组独立资源组成，每个资源（如 I/O、内存、ALU）占用一个字。

编码的另一个方面是：它是直接编码，还是间接编码（图 21-11b）。对于间接编码，一个字段用于确定另一个字段的解释。例如，考虑一个 ALU 能执行 8 种不同的算术运算和 8 种不同的移位操作。一个 1 位的字段可用于指示使用的是移位操作还是算术运算；一个 3 位的字段表示操作码。这种技术通常意味着两级译码，增加了传播延迟。

图 21-12 是这些概念的一个简单示例。假设处理器有一个累加器和几个内部寄存器，比如程序计数器和用于 ALU 输入的暂存寄存器。图 21-12a 显示了一个高度垂直的格式。前 3 位表示操作类型，接下来的 3 位对操作进行编码，最后 2 位选择一个内部寄存器。图 21-12b 是更为水平的方法，但仍然使用了编码。本例中，不同功能出现在不同的字段中。



图 21-12 简单机器的两种微指令格式

## 14. 指令和微指令的异同

### （1）相同点

- 都是指令的有序序列，驱动计算机执行操作；
- 都包含操作码/控制域、地址域；
- 都有取指、执行的周期流程；
- 都支持顺序执行和分支跳转。

### （2）不同点

| 维度   | 机器指令 (Instruction)                     | 微指令 (Microinstruction)           |
|------|----------------------------------------|----------------------------------|
| 层级   | 程序员可见, 属于指令集架构 (ISA) 层                 | 程序员不可见, 属于 CPU 硬件控制层             |
| 存储位置 | 主存储器 (内存)                              | 控制存储器 (CM, CPU 内部的高速 ROM)        |
| 功能   | 完成一个完整的运算 / 访存 / 跳转 / 逻辑功能             | 完成一个不可再分的原子微操作 (如寄存器传送、ALU 运算控制) |
| 对应关系 | 1 条机器指令 $\rightarrow$ 1 段微程序 (包含多条微指令) | 1 条微指令 $\rightarrow$ 一同时执行的微操作   |
| 长度   | 可变长度 (CISC 架构) / 固定长度 (RISC 架构)        | 固定长度 (由 CPU 硬件设计决定)              |
| 寻址方式 | 支持多种复杂寻址方式 (立即、间接、寄存器、基址、变址等)          | 仅支持微程序内的简单寻址 (顺序、条件转移、无条件转移)     |
| 使用者  | 编译器、汇编程序员、高级语言程序员                      | CPU 硬件控制层、微程序设计者                 |

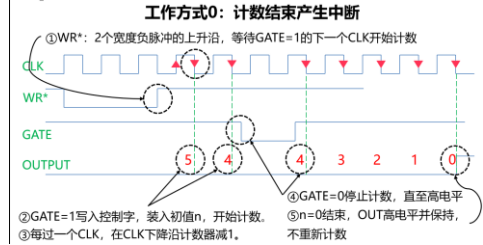
## 可编程计数器/定时器 8253：

该芯片的最高计数频率为 2MHz，可用于产生各种定时信号，也可用于对外部计数。寻址方式：右图

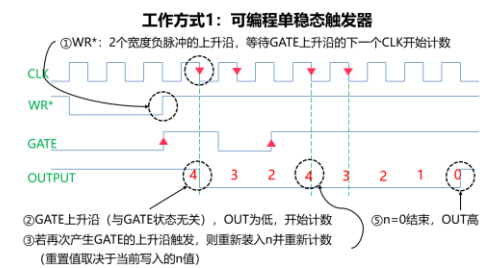
控制字格式：（端口地址 43H）：计数器通道 X 的端口地址为 4XH

| D7         | D6         | D5        | D4       | D3 | D2 | D1 | D0 |
|------------|------------|-----------|----------|----|----|----|----|
| 00: 选计数器 0 | 00: 计数器锁存  | 000: 方式 0 | 0: 二进制   |    |    |    |    |
| 01: 选计数器 1 | 01: 对低字节读写 | 001: 方式 1 | 1: BCD 码 |    |    |    |    |
| 10: 选计数器 2 | 10: 对高字节读写 | X10: 方式 2 |          |    |    |    |    |
|            | 11: 先读写后低高 | X11: 方式 3 |          |    |    |    |    |
|            |            | 100: 方式 4 |          |    |    |    |    |
|            |            | 101: 方式 5 |          |    |    |    |    |

计数初值：BCD 码 (D0=1) 装入 0000H，最多计数 10<sup>4</sup> 次；二进制 (D0=0) 装入 0000H，最多计数 2<sup>16</sup>=65536。（3 个计数器均为 16 位）（输出方波方式 3，输出负脉冲序列用 2，输出单个负脉冲用 1，定时发送用 2、3）二进制 65536—0000H 256-000H BCD1000-00H 计数初值=输入频率/要求频率方式 0 (计数结束产生中断)：小结：☆1、写信号后沿上升沿经一个 CLK 上升下降将 n 值装入计数器；☆2、CLK 下降沿，计数器减一；☆3、软件启动，不自动重复计数，只有写入另一个计数值时，即刻触发按新值计数（计数过程中可重新置数）；☆4、装入初值后 OUT 变低电平，计数结束 OUT 输出高电平；☆5、计数过程中，GATE 端应保持高电平。方式 0 时序图 ↓



方式 1 (可编程单稳态触发器) 小结：1、门控信号 GATE 的上升沿启动计数（硬件启动），GATE 的上升可重复触发计数，在计数过程中，若 GATE 变低，不影响计数；2、装初值后 OUT 端变低电平，计数开始 OUT 端变为低电平，计数结束后又变高；3、计数过程中，GATE 再次上升沿触发，单负脉冲拉宽为两次计数过程之和，脉冲宽度为  $t_c \times N$  ( $t_c$  为时钟周期)；方式 1 时序图 ↓



方式 2 (分频器)：GATE=1 时，写入控制字，OUT 端输出高电平为起始电平，装入计数初值 n，开始计数；每一个计数脉冲下降沿 n 减 1，当 n 减至 1 时，OUT 变低，n 减为 0 时，OUT 变高，产生一个与时钟脉冲周期一样宽的负脉冲；接着自动装入 n 连续计数，输出频率为  $f_{clk}/n$ ；计数过程中，允许重新装入新的 n 值，下一个计数周期按新的 n 值计数。GATE=0，停止计数，并强迫 OUT 输出高电平，在 GATE 变为高电平后，重新装入 n 值，开始计数。

方式 2 时序图 ↓（正脉冲宽度实际为  $t_c \times (n-1)$ ）负脉冲序列

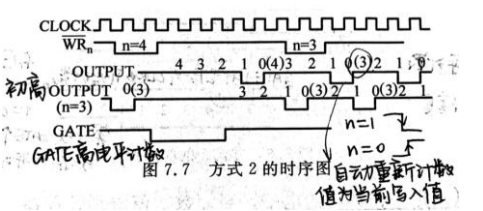
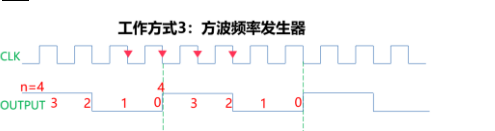


图 7.7 方式 2 的时序图

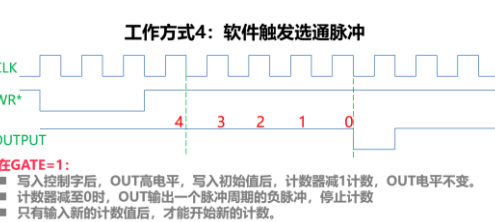
## III] 方式 3 (方波频率发生)：(方式 3 时序图)



在 GATE=1：输出频率为  $f_{clk}/n$  的方波。

- 写入控制字后，OUT 端输出低电平作为起始电平，装入计数值 n 后，变为高电平。
- n 为偶数，每个时钟脉冲下降沿 n 减 1，至 n/2 后，电平变为低电平，并继续减 1 计数至 0，然后改变 OUT 电平，重新装入 n，开始计数。
- n 为奇数，输出高电平宽度为  $(n+1)/2$ ，低电平宽度为  $(n-1)/2$  的方波。
- GATE=0：停止计数，并强迫 OUT 输出高电平，在 GATE 变高后，重新将 n 装入，开始计数。

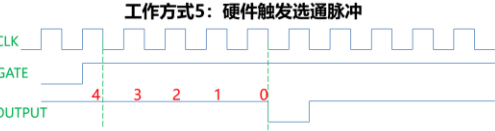
方式 4 (软件触发选通脉冲)：(方式 4 时序图) GATE 高电平+手动输入)



在 GATE=1：写入控制字后，OUT 高电平，写入初值 n 后，须等待 GATE 上升沿才计数。计数器减至 0 时，OUT 输出一个脉冲周期的负脉冲，停止计数。只有当 GATE 的上升沿到来后才再次开始计数。

在 GATE 在计数中改变：计数过程中，若 GATE 变低，停止计数，变高后，重新将 n 装入，开始计数。

方式 5 (硬件触发选通脉冲)：(方式 5 时序图) GATE 上升沿+自动置入)



在 GATE 产生一个上升沿：写入控制字后，OUT 高电平，写入初值 n 后，须等待 GATE 上升沿才计数。计数器减至 0 时，OUT 输出一个脉冲周期的负脉冲，然后 n 值自动装入计数器，但须等 GATE 的上升沿到来后才再次开始计数。计数过程中，若 GATE 变低电平，不影响计数。如果是计数结束后，再一次产生 GATE 的上升沿，则再次计数。

初始化编程：在使用 8253 芯片时，首先要对其进行初始化编程，包括写入控制字和写入计数值两个步骤。

第一步，写入控制字。任一计数器的控制字都要从 8253 的控制口地址写入，控制哪个计数器由控制字的 D7D6 位来决定，写入控制端口

第二步，写入计数初值。计数初值经由各计数器的端口地址写入。若控制字里规定只写低八位，则写低八位，高 8 位自动置 0。若控制字里规定只写高八位，则写高八位，低 8 位自动置 0。若是 16 位计数值，则分两次写入，先写低 8 位，再写高八位。

e.g. 1MHz 定时 1s (单次)：计数器 0 工作方式 3，计数值 10000=2710H，先传 10 再传 27；计数器 1 工作方式 1 (周期 1s 为方式 3)，计数值 100-64H；1MHz 进 clk0，100Hz 从 OUT0 出 clk1，OUT1 出 1s

例 1：三个计数器端口地址为：40H，41H，42H，控制寄存器端口地址为：43H；时钟 1.19MHz。

计数器 0：电子时钟基准，工作于方式 3，计数器值预置为 0 (即 65536)，即输出的方波频率为：1.19/65536=18.21Hz，即每隔 55ms 产生一次中断请求。程序：

```
MOV AL, 36H ; 36H=00110110B
OUT 43H, AL ; 写控制字
MOV AL, 0 ; 预置计数值
OUT 40H, AL ; 先写低字节
OUT 40H, AL ; 再写高字节
```

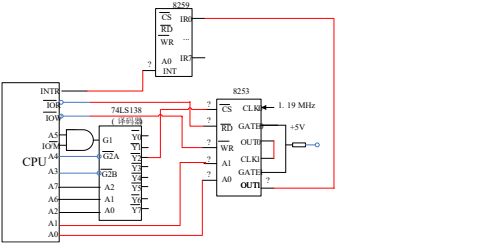
计数器 1：动态 RAM 定时刷新，工作于方式 2，计数器值预置为 18，即输出的频率为：1.19/18=66.1kHz，即每隔 15.1us 产生一次动态刷新请求信号。程序：

```
MOV AL, 54H ; 54H=01010100B
OUT 43H, AL ; 写控制字
MOV AL, 12H ; 预置计数值
OUT 41H, AL ; 写低字节
```

计数器 2：扬声器音调，工作于方式 3，计数器值预置为 1331 (533H)，即

输出的频率为：1.19/1331=894Hz。程序：  
MOV AL, B6H ; B6H=10110110B  
OUT 43H, AL ; 写控制字  
MOV AL, 33H ; 预置计数值  
OUT 42H, AL ; 先写低字节  
MOV AL, 05H ; 预置计数值  
OUT 42H, AL ; 再写高字节  
若计数值 > 65536，则通过两个计数器级联（前一个 OUT 接后一个 CLK）

例 2，图中设计采用 8253 计数器和 8259A 中断芯片，8253 的三个计数器端口地址为 60H~62H，控制寄存器端口地址为 63H，8253 每隔 10s 通过 out1 端口向 8259a 的 IR0 端口向 CPU 请求一次中断，请完成图中的硬件连线（？处），并写出 8253 的初始化程序。



此类问题分析如下：★8253 的 RD\WR 一般接 CPU 对应的 OR\OW；★8253 的 A0\A1 负责选择计数器（四个端口依次为 00\01\10\11），接 CPU 对应的接口（一般是 A0\A1 但不一定）；★8253 的 CS 对应的数字 Yi 决定 CPU 的 A7\A6\A2 设置（本题为 010）；★74138 的 G1 对应的 A5 和 I0\I1 设置为 1，G2A\G2B 对应的 A3\A4 设置为 0；★因此 8253 四个端口对应 CPU 的 A7~0 为 01100000B~01100011B 即为 60H~63H；★8253 的 OUT0 可以单独输出，计数值：n=1.19MHz/(1/10)=11900000/65536 (n 最大值 65536)，也可以接 CLK1，用 2 个计数器进行 2 次分频 n=n0×n1；★8253 的 OUT1 接 8259 的 IR0 (最高级中断)，8259 的 INTR 接 CPU 的 INTR，即 8253 输出 OUT1 时，CPU 中断；

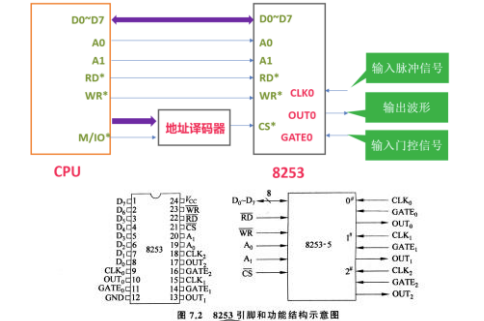
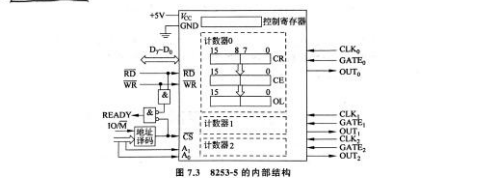


图 7.2 8253 引脚和功能结构示意图

片内地址：A0、A1：地址线，用于选择 5 个计数器中的一个及选择控制寄存器。  
RD：读控制信号，低电平有效。  
WR：写控制信号，低电平有效。  
CS：片选端，低电平有效。  
CLK<sub>n-1</sub>：计数器 0、1、2 的时钟输入端。  
GATE<sub>n-1</sub>：计数器 0、1、2 的门控脉冲输入端，将外部设备送入门控脉冲。  
OUT<sub>n-1</sub>：计数器 0、1、2 的输出端，由它接至外部设备以控制其启停。

1. 内部结构  
8253-5 的内部结构如图 7.3 所示。它由 3 个独立结构完全相同的 16 位计数器和 1 个控制寄存器。



在每个计数器内部，又可分为计数初值寄存器 (CR)、计数执行部件 (CE) 和输出锁存器 (OL) 这 3 个部件。它们都是 16 位寄存器，也可以作 8 位寄存器来用。在计数器工作时，通过过程给初值寄存器 (CR) 送入初始值，该值再送入执行部件 (CE) 作减 1 计数，而输出锁存器 (OL) 则用来锁存 CE 的内容，该内容可以由 CPU 进行读/写操作。

可编程并行通信接口芯片 8255A:

该芯片有三组 8 位并行可编程端口 A、端口 B、端口 C，提供不同应用所要求的和外部设备进行连接的方法，它们既能够在同步传送方式，又能够在异步传送方式，既可以作为输入端口，也可以作为输出端口。

(其中端口 C 可作两个 4 位输入/输出端口；并可以配合 A、B 口做控制/状态信号)

| 四个端口代表端口 A、B、C 控制口 |    |    |    |    |       |  |  |
|--------------------|----|----|----|----|-------|--|--|
| A1                 | A0 | RD | WR | CS | 操作    |  |  |
| 0                  | 0  | 0  | 1  | 0  | 读端口 A |  |  |
| 0                  | 1  | 0  | 1  | 0  | 读端口 B |  |  |
| 1                  | 0  | 0  | 1  | 0  | 读端口 C |  |  |
| 0                  | 0  | 1  | 0  | 0  | 写端口 A |  |  |
| 0                  | 1  | 1  | 0  | 0  | 写端口 B |  |  |
| 1                  | 0  | 1  | 0  | 0  | 写端口 C |  |  |
| 1                  | 1  | 1  | 0  | 0  | 只写控制字 |  |  |

控制字: 根据 D7 位的不同，控制字分为两类  
方式选择控制字 (D7=1)、C 端口置位复位控制字 (D7=0)

方式选择控制字 (D7 = 1) 【★查询方式 0 之前请务必先查询该表★】

| D7 | D6      | D5      | D4      | D3        | D2        | D1      | D0        |
|----|---------|---------|---------|-----------|-----------|---------|-----------|
| 1  | A 口工作方式 | A 口工作方式 | C 口工作方式 | B 口工作方式   | B 口工作方式   | C 口工作方式 | C 口工作方式   |
| 1  | 0       | 1       | 1/0     | 高 4 位 I/O | 低 4 位 I/O | 1/0     | 低 4 位 I/O |

D6/D5: A 口工作方式 00 方式 0; 01 方式 1; 1X 方式 2  
D2: B 口工作方式 0 方式 0; 1 方式 1  
I/O: 输入输出方式 0 输出; 1 输入

C 端口置位复位控制字: 用于设置 C 口的某一位 D7=0

| D7 | D6 | D5 | D4   | D3   | D2   | D1   | D0       |
|----|----|----|------|------|------|------|----------|
| 0  | 未用 | 未用 | 位置选择 | 位置选择 | 位置选择 | 位置选择 | 1 置/0 复位 |

| D7 | D6 | D5 | D4  | D3 | D2 | D1 | D0  |
|----|----|----|-----|----|----|----|-----|
| 0  | ×  | ×  | ×   | B2 | B1 | B0 | I/O |
| 1  | 0  | 1  | 1/0 | 1  | 1  | 1  | ×   |

| 特征位 | 位置 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|-----|----|---|---|---|---|---|---|---|---|
| B0  | 0  | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 |
| B1  | 0  | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 1 |
| B2  | 0  | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 |

方式 0: 基本输入/输出工作方式 (可用于无条件传送或查询式传送)

【★查询方式 0 之前请务必先查询方式选择控制字表★】

特点: 在该方式下 8255 提供了最基本的输入/输出工作方式, 它没有“握手联络”信号, 微处理器只要对端口进行简单的读和写就能完成数据的输入和输出操作。

☆1. 在该方式下, 8255A 可以定义为两个 8 位端口 AB, 两个 4 位端口 C。☆

2. 任何一个端口都可用作输入或输出。☆3. 由 A 口、B 口、C 口高 4 位与 C 口低 4 位 4 组合成 16 种不同的输入输出状态。☆4. 可用于同步传送和查询传送。在查询传送时, 通常将 C 口的某些位规定为控制或状态信号端口, 具体的功能可以由程序指定。

控制字格式如下:

| D7 | D6 | D5 | D4 | D3      | D2 | D1 | D0      |
|----|----|----|----|---------|----|----|---------|
| 1  | 0  | 0  | A  | C 高 4 位 | 0  | B  | C 低 4 位 |

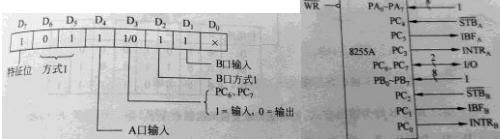
见最后  
例: 8255 的四个端口为: 80H~83H。A 口工作方式 0, 输出, B 口工作方式 0, 输入。则指令 MOV AL, 10000010B ; OUT 83H, AL;

方式 1: 选通输入/输出工作方式 (异步, 可用于查询式传送或中断式传送)

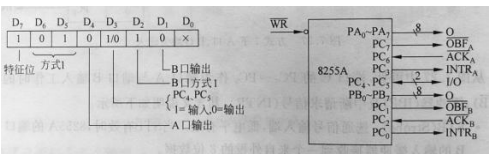
特点: A 口、B 口作为 8 位数据端口, 可以工作于选通输入/输出方式, 当要在联络信号的控制下才能完成 I/O 操作; C 口的 6 根 I/O 线作为 A 口、B 口的联络信号; C 口余下的两根 I/O 线仍用来输入输出数据。

端口 A 和 B 均为输出方式控制字格式:

| D7 | D6 | D5 | D4 | D3           | D2 | D1 | D0 |
|----|----|----|----|--------------|----|----|----|
| 1  | 0  | 1  | 1  | PC6 PC7: I/O | 1  | 1  | X  |



| D7 | D6 | D5 | D4 | D3           | D2 | D1 | D0 |
|----|----|----|----|--------------|----|----|----|
| 1  | 0  | 1  | 0  | PC4 PC5: I/O | 1  | 0  | X  |



| D7 | D6 | D5 | D4 | D3 | D2 | D1 | D0 |
|----|----|----|----|----|----|----|----|
| 1  | 0  | 1  | 1  | ↓  | 1  | ↓  | X  |

D4: A 口方式 1: 输入; 0 输出  
D3: PC6、PC7 1: 输入; 0 输出  
D1: B 口方式 1: 输入; 0 输出

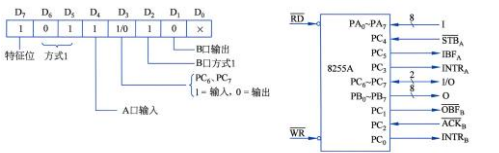


图 7.30 方式 1 下端口 A 输入、端口 B 输出

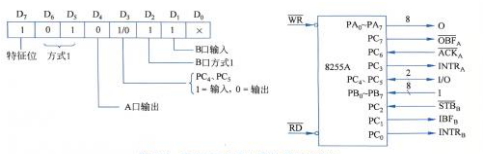


图 7.31 方式 1 下端口 A 输出、端口 B 输入

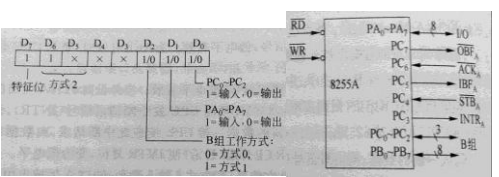
例: 8255 的四个端口为: 80H~83H。要对 PC7 置 1, 对 PC3 置 0。则指令:  
MOV AL, 0FH ; OUT 83H, AL; PC7 置 1: 00001111B  
MOV AL, 06H ; OUT 83H, AL; PC3 置 0: 00000110B

方式 2: 选通双向传输 (异步, 仅适用于端口 A)

特点: 双向方式中 A 端口在某一时刻工作在输入方式, 在另一时刻又可以工作在输出方式; 需要有两组 6 位联络信号, 分别与输入和输出设备进行联络, 联络信号由 C 端口中的 5 个數位提供, 其中输入和输出的中断请求信号合并为一个; 联络信号由 C 端口提供, C 端口只有 8 位, 只能为一个端口提供联络在方式 2 下工作, 因此在 8255A 中只有 A 组控制的 A 端口能够工作在双向方式 控制字格式如下 ↓

| D7 | D6 | D5 | D4 | D3 | D2 | D1 | D0 |
|----|----|----|----|----|----|----|----|
| 1  | 1  | 1  | X  | X  | ↓  | ↓  | ↓  |

D2: B 口工作方式 1: 方式 1 ; 0: 方式 0  
D1: PA0 至 PA7 方式 1: 输入 ; 0: 输出  
D0: PC2 至 PC2 方式 1: 输入 ; 0: 输出



时序图: 太复杂, 编者看不懂, 故不放了

例题 (2012 年第五题) 下图为 8086 系统的一个接口电路, 有一片 8259, 一片 8255A 及 8 个发光二极管, 通过按钮开关可以产生中断请求信号。要求通过 8259 产生中断请求, 在中断服务程序中通过 8255A 的 A 口输出控制 8 个发光二极管轮流点亮。8259 的端口地址为 40H、41H, 中断类型号为 80H~87H。工作方式: 边沿触发, 一般单级, 一般结束中断, 非缓冲方式; 8255A 按基本输入输出方式工作, 端口地址为 60H~63H。  
1) 请写出 8259 的初始化程序, 并设置中断屏蔽字 OCW1:

2) 写出 8255A 初始化程序, 给 8255A 的 A 口设置初值, 将初值保存到“INIT”单元;  
3) 请编程设置中断向量, 假设中断服务程序入口处的标号为“LIGHT\_INT”;  
4) 请写出对应的中断服务程序 (包括开中断、8255A 的 A 口输出控制 8 个发光二极管轮流点亮、给 8259 发 EOI 命令、中断返回)。

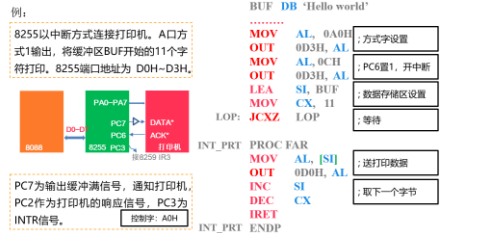
1) 8259 初始化 (4 分)  
MOV AL, 00010011B; 写 ICW1  
OUT 40H, AL; 80H: 中断类型号  
MOV AL, 41H, AL; 写 ICW4  
OUT 41H, AL; 0 ; 写 OCW1 屏蔽字, 允许 IR3 中断  
MOV AL, 41H, AL; 41H  
OUT 41H, AL; 11110111B  
AND AL, 11110111B; 写 OCW1 屏蔽字, 允许 IR3 中断  
OUT 41H, AL;

2) 8255A 初始化 (4 分)  
MOV AL, 80H ; 写控制字  
OUT 63H, AL; MOV DX, 63H  
MOV AL, 01H (PA0~PA7 对应端口)  
OUT 60H, AL; 给 8255A 的 A 口设置初值, D1 亮, 其它不亮。  
INIT, INT 21H

3) 设置中断向量 (4 分)  
CLI  
MOV AX, SEG LIGHT\_INT  
MOV DS, AX  
MOV DX, OFFSET LIGHT\_INT  
MOV AL, 83H 对应 IR3  
MOV AH, 25H  
INT 21H  
STI

4) 中断服务程序 (4 分)  
LIGHT\_INT: STI ; 开中断  
PUSH AX  
MOV AL, INIT  
ROL AL, 1 ; 控制发光二极管轮流点  
OUT 60H, AL ; 从 A 口送出  
MOV INIT, AL  
MOV AL, 20H; 设置 8259OCW2, EOI=1, 结束 8259 中断  
OUT 40H, AL ; 发出 OCW2  
POP AX  
IRET ; 中断返回

(每按一次进入中断程序)



8255与CPU连接图



(一) A、B 均为输入方式  
(1) STB\*: 选通信号输入端, 由外设提供使 8255A = 0 时, 外部数据经 PA0~PA7 输入 8255A 的 A 口缓冲器。  
(2) IBF\*: 输入缓冲器满信号 (输出), 是 STB\* 的有效响应信号, 表示外设的数据已输入 A 口缓冲器中, CPU 完成读入操作后, 在 RD\* 上升沿之前复位 (变低电平)。  
(3) INTR\*: 中断请求信号 (输出) 当 STB\*, INTE, IBF 全为高电平时 INTR 有效, 向 CPU 申请中断, 表示数据缓冲器已有一个新数据, 若响应则读入该数据, 并由 RD\* 下降沿复位 (变低电平)。  
(4) INTE\*: 中断允许信号 (输入) 下降沿触发, =1 表示允许; 由 A/B 的 INTR 通过对相应的 STB\* 置位 1 或复位 0 来控制。

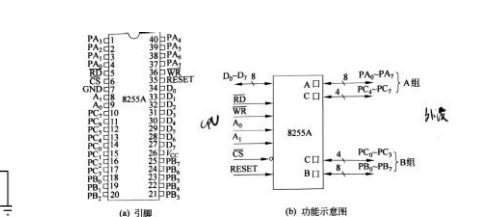


图 7.24 8255A 引脚和功能示意图

CS\*: 片选端, 低电平有效。  
RESET\*: 复位信号, 高电平有效。8255A 复位后, 所有 I/O 均处于输入状态。  
A 口: 8 位数据输入缓冲器和 8 位数据输出缓冲器/缓冲器。  
B 口: 8 位数据输入缓冲器和 8 位数据输出缓冲器/缓冲器。  
C 口: 8 位数据输入缓冲器和 8 位数据输出缓冲器/缓冲器。  
实际使用时, 可以把 A 口、B 口、C 口分成两个控制组, 即 A 组和 B 组。A 组控制电路由端口 A 和端口 C 的高 4 位 (PC4~PC7) 组成, B 组控制电路由端口 B 和端口 C 的低 4 位 (PC0~PC3) 组成。

8255A 的内部结构框图如图 7.25 所示。其中, 各个部件的具体组成与功能如下所示。

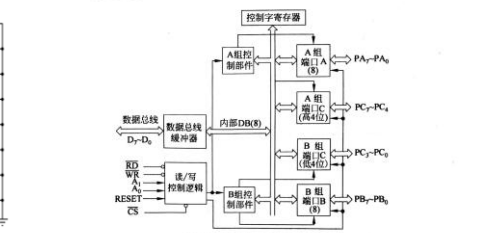
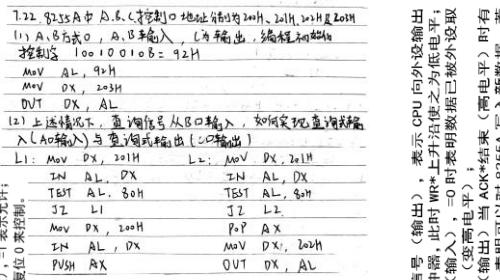


图 7.25 8255A 的内部结构框图

7.24 如果需要 8255A 的 PC3 输出连续方波, 如何用 C 口的置位与复位控制命令编程实现它?

```
MOV DX, 203H
L: MOV AL, 06H ; PC3 清零 0000 0110
OUT DX, AL
NOP
MOV AL, 07H ; PC3 置 1 0000 0111
OUT DX, AL
NOP
JMP L
```



方式 0 最基本的输入/输出工作方式, 其控制字格式如图 7.26 所示。



图 7.26 方式 0 的控制字格式

(二) A、B 均为输出方式  
(1) OBF\*: 输出缓冲器满信号 (输出), 表示 CPU 向外设输出的数据已放入相应输出缓冲器, 此时 WR\* 上升沿使之变为低电平;  
(2) ACK\*: 外设应答信号 (输入), =0 时表示输出数据已被外设取走, 同时 ACK\* 使 OBF\* 复位 (变高电平)。  
(3) INTR\*: 中断请求信号 (输出) 当 ACK\* 结束 (高电平) 时有效, 向 CPU 发送中断请求, 表明可以对 8255A 写入新数据, 若响应则由 WR\* 上升沿来复位 (变低电平)。  
(4) INTE\*: 中断允许信号, =1 表示允许; 由 A/B 的 INTR 通过对相应的 ACK\* 置位或复位来控制。